

Sustain



System design, implementation, and the path to public release

A practical guide to the application, the engineering behind it, and the tradeoffs you can explain in a software engineering interview.

Reviewed 23 September 2026

Source: main, commit 088f60cb8b001cfa669a21f3250426c8a0067d20. The application working tree was clean at the start of this review. Repository and native identifiers still use HealthApp.

Sustain is an iPhone-first wellness tracker built for personal daily use. It combines manual and Apple Health readings, nutrition, training, fluids, goals, reminders, and private progress photos. Its most substantial engineering is in preserving the meaning of history and making uncertain saves recoverable.

The defensible design story: build something useful and maintainable for one person, establish sound ownership and consistency boundaries, and add capacity infrastructure only when measured demand justifies it.

This report explains what is implemented, how it works, why its choices are reasonable, where its guarantees stop, and what a responsible beta and public launch would require. It does not claim measured production capacity or clinically validated estimates.

Documentation-only review. No application behavior changed, services deployed, paid model requests made, or real user data modified. Code references and verification evidence are indexed at the end.

Contents and reading paths

Pages	Topic
3-4	Original goals, evidence standard, current feature boundaries
5-11	First principles, architecture, stack, storage, data model
12-16	Authentication, Face ID, security, entry, onboarding, Profile
17-23	Summary, streaks, vitals, synchronization, HealthKit, training, planner
24-30	Meals, labels, recipes, barcodes, AI estimates, fluids, goal helpers
31-35	Reminder configuration, scheduling, routing, photos, export
36-39	Retry transactions, concurrency, offline behavior, account deletion
40-43	Design decisions, alternatives, and reconsideration triggers
44-46	Verification, findings, and prioritized improvements
47-51	Deployment architecture, beta, public release, compatibility, operations
52-55	Workload scenarios, bottlenecks, costs, staged scaling recommendation
56-58	Interview answers with deeper explanations and follow-up questions
59-60	Alphabetized glossary
61-64	Code evidence map, verification records, authoritative references

How to read it

For a first pass, read pages 3-10, then follow the features you use daily. For a system-design interview, concentrate on the saving examples, alternative designs, deployment plan, and scaling scenarios. For launch planning, start with the findings and release gates.

Deeper dive paragraphs unpack mechanisms you can draw or explain at a whiteboard. An unfamiliar term is introduced in the narrative before being used technically; the glossary is a reminder rather than a substitute for that explanation.

Evidence IDs such as C08 point to precise repository paths in the appendix. R01-style IDs identify official documentation. These distinguish what the code does from platform behavior and recommendations. Numerical workload scenarios are planning assumptions, never benchmark results.

3. Goals and evidence discipline

What the project was trying to achieve

Your stated starting requirement was a useful personal app, not a large public service. One developer needed to build and maintain logging, history, goals, and native iPhone integrations. Simplicity meant limiting the number of systems to operate, while still protecting your records from lost saves and accidental overwrites.

Your second stated motivation was experience with widely used engineering technologies. React components, TypeScript, API integration, state ownership, SQL, and tests offer transferable skills. That career motivation complements the practical fit of a shared TypeScript client and native adapters; it does not prove that other frameworks would be inferior.

Future accounts were a consideration, not a measured requirement. Account ownership, access policies, stable identifiers, and explicit save contracts make growth easier. They do not establish throughput, uptime, or a supported population size.

How claims are classified

Label	What it establishes
Implemented and reachable	A current route/control connects to the inspected implementation.
Present but inactive	Code/schema exists for compatibility or future work; current UI does not expose it.
Verified this review	A named local check ran against the reviewed source snapshot.
Historical deployment	Repository records report a hosted deployment/check; this review did not repeat it.
Independently verified deployed	Requires fresh inspection of the running service. No such claim is made here.
Proposed or unknown	A recommendation, or a question the available evidence cannot settle.

The attached reference PDF was read and matched the repository copy by SHA-256. It is background context; current routes, repositories, migrations, tests, and configuration are the evidence for implementation claims.

Rationale attribution: your personal-use and learning goals are user-stated. Architecture Decision Records (ADRs) document selected decisions. Other comparisons are retrospective engineering analysis. No undocumented benchmark, historical experiment, or team discussion is invented.

C01, C02. Snapshot: main / 088f60c; 36 SQL migration files; five server-function entry points. Counts describe source, not hosted deployment state.

4. Current product boundaries

Area	Reachable behavior and boundary
Account	Email/password signup, sign-in, verification/recovery, optional Face ID. Remember me retains identity, not a persisted Auth session.
Setup and Profile	Authenticated onboarding; names, units, goals, appearance, health import, sync recovery, export, security, deletion.
Summary	Twelve widget types, eight defaults, local layouts, current-week goal streaks, trends and calorie calendar.
Vitals	Dated weight and paired blood pressure (BP), optional pulse, manual editing/deletion, durable local queue.
Apple Health	Read-only weight and BP imports with nearby pulse association. No workouts, sleep, steps, or heart-rate-variability feed.
Training	Detailed lifting and manual cardio; local drafts; versioned online saves; one-session AI planning.
Food	Meals, reusable labels, recipes, barcode lookup, reviewed photo/text AI estimates; immutable consumed snapshots.
Fluids and goals	Beverage volume/classification, calorie/protein/fluid/weight/BP targets, persistent calorie/fluid helper estimates.
Reminders	Five routine categories plus medication/supplement group and named custom reminders. Device-local schedules and completion history.
Photos	Private processed images linked to weight entries, at most three new attachments per entry, signed viewing URLs.
Soon	A designed Coming Soon placeholder; no hidden functional product is established.

Boundaries that change the explanation

The four offered streaks are **calorie target, protein target, fluid target, and training consistency**. There is no food-day-complete control and no current logging streak. Calorie evaluation uses nonempty recorded intake, so an under-target check is not proof of complete eating records.

The current AI planner is a questionnaire that opens editable workout drafts. Stored Coach threads and a broader compatible backend do not make the current product a general chat interface. Likewise, a phone route redirects to email sign-in; phone-related code does not establish an exposed SMS login flow.

Cloud records are shared across devices; layouts, reminders, completion history, and drafts generally are not. A draft surviving offline is different from a completed workout being accepted offline. Full process termination returns to sign-in, even with remembered identity.

C03-C20. Inventory rebuilt from routes, widget registry, current questionnaire, account screens and repositories; compatibility code is identified in the relevant chapters.

5. First principles: a save crosses boundaries

Imagine entering yesterday's weight on your phone. Several separate responsibilities are involved, even though the interaction feels like one button.

The frontend is the part you interact with: screens, input fields, navigation, charts, and feedback. Sustain's React Native code runs on the device. It can validate obvious mistakes immediately and retain work locally, but a modified client could bypass those checks.

The backend is the remote system that coordinates accounts and shared records. Sustain uses Supabase services and PostgreSQL. The backend must independently check identity, ownership, and data rules, because it cannot trust a client simply claiming a particular user ID.

An application programming interface (API) is an agreed request/response contract between components. A repository calling `commit_health_mutation` sends an operation identifier and a structured request. It receives an accepted result, conflict, or error. That contract lets a screen avoid knowing every database detail.

A database stores structured records with relationships and rules. A workout session and its sets are rows. **Object storage** holds larger files, such as JPEG images. A photo's database row describes who owns it and where its bytes live; those are separate resources.

Why a spinner cannot tell you what happened

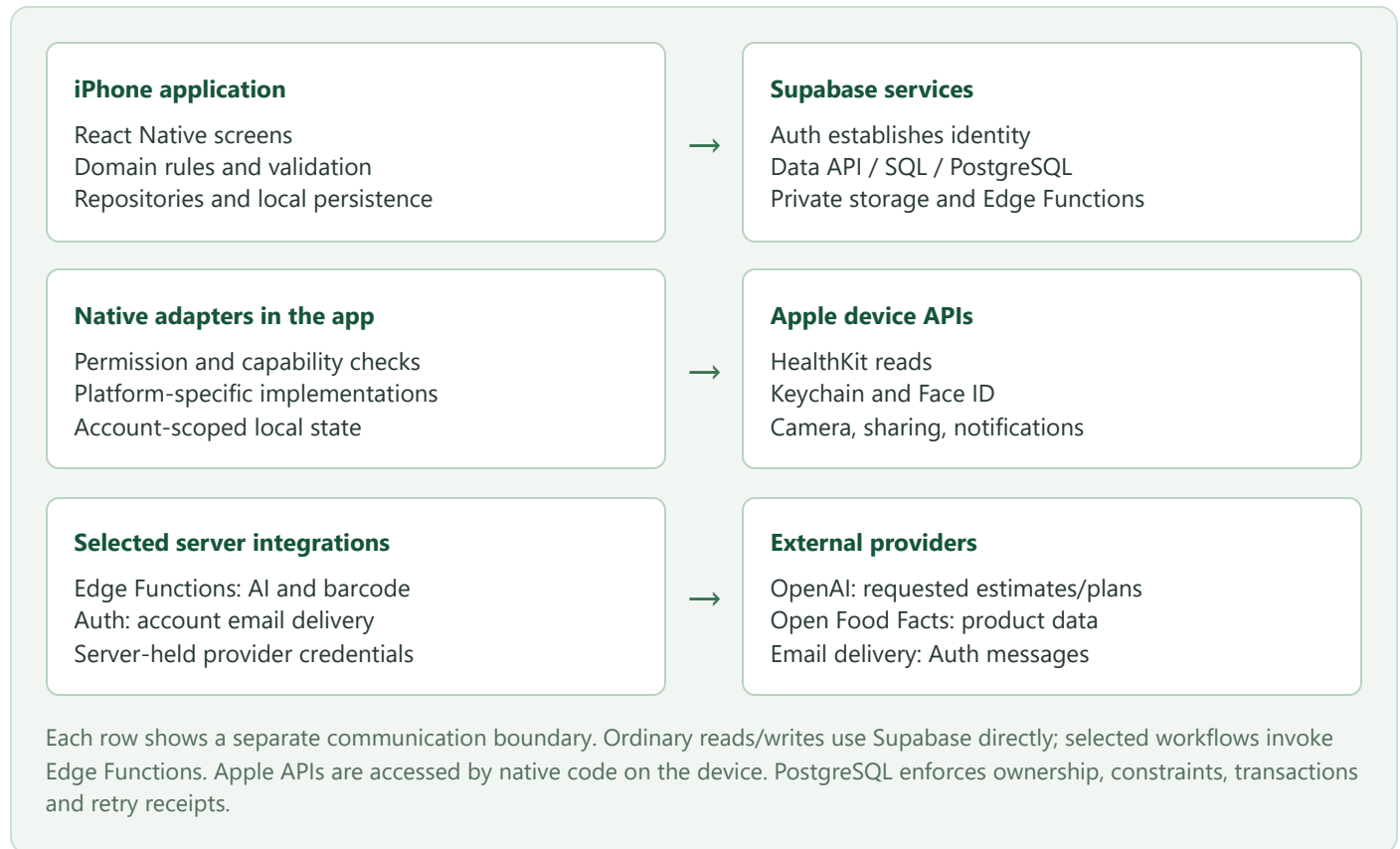
If the network disappears, the phone may not know whether the server saved the meal. A timeout means the response was not confirmed, not necessarily that the save failed. Sustain persists an operation identity and request so it can ask again without creating another meal.

A **transaction** groups database changes so they either commit together or roll back. It can protect the meal rows and retry receipt together. It cannot also guarantee that a native notification was delivered, an external model answered, or a separately uploaded file was removed.

Think in boundaries: what is durable on the device, what is committed on the server, and what remains uncertain? Most of Sustain's reliability code exists to answer those three questions precisely.

C04, C08, C09. These definitions describe the actual repository and mutation paths, not an additional API server or queue.

6. Architecture and trust boundaries



The internal layers

Screens own interaction state: selected tabs, busy flags, field text, and explicit review. Pure domain modules implement calculations and validation without rendering. Repositories map domain values to database columns and save contracts. Platform adapters select native or browser implementations for storage, images, notifications, and HealthKit.

The structure is a feature-modular client plus a managed backend. It does not contain a separate always-running Express service, microservice fleet, Redis cache, or distributed message broker. Local promise serialization is not a distributed queue.

Where trust changes

Device inputs and model/provider responses are untrusted. TypeScript helps developers but cannot verify a network payload. Zod checks runtime shape; database constraints and ownership checks enforce final write rules. A service-role credential can bypass ordinary account policies, so it belongs only in reviewed server code.

This shape keeps infrastructure manageable for one developer while preserving meaningful boundaries. The cost is distributed business logic across client models, repositories, SQL functions, and a few Edge Functions. The evidence map is useful precisely because no single file explains every invariant.

C01, C04, C08, C09, C21. See pages 40-43 for alternatives; diagram is an implementation map, not a capacity claim.

7. The client stack, explained

React is a way to describe a user interface as components whose output follows state. A goal editor can render a saved value, an input draft, and an error as distinct states. React itself is not the mobile operating system or the backend.

React Native applies that component model to native platform views and modules. A Sustain view or Text is not simply an HTML element inside a browser. React Native Web provides a separate browser rendering path for useful previews and tests.

Expo supplies the application toolchain, native module packages, configuration, development client, and build services. Sustain uses Expo SDK 57, React Native 0.86.3, React 19.2.3, TypeScript 6, and Expo Router as declared by its manifests. The lockfile fixes the installed dependency graph; semver ranges alone do not identify every installed patch.

Expo Router maps files to navigation destinations. Layouts implement public/private route boundaries, tabs, nested Profile screens, and entry gates. File routing does not authorize database access; backend rules remain necessary.

Developer-time types versus runtime validation

TypeScript catches incompatible assignments while developing. It disappears during compilation. If an external service returns calories as an invalid string, a TypeScript interface cannot repair it. Zod schemas parse actual runtime values and reject malformed input, drafts, and responses. SQL constraints independently protect stored records.

State and visual tooling

Hooks, context providers, refs, and feature-local models coordinate state. There is no required Redux or query-cache framework in the current save design. React Native SVG draws charts and the branded symbol effects; animation, gesture, safe-area, and screen packages handle platform presentation. Expo haptics and splash-screen support entry feedback.

The transferable skills are component composition, state ownership, asynchronous effects, TypeScript modeling, validation, API contracts, and testing. Native keyboard behavior, lifecycle transitions, permissions, signing, and accessibility still require platform-specific knowledge. The benefit is shared structure and domain code, not identical behavior everywhere.

C01, C03, C05. Versions are source declarations. Provider tooling does not implement Sustain's navigation guards, form semantics, retry state, or native acceptance tests.

8. Backend and platform responsibilities

Supabase is the managed platform; **PostgreSQL** is the relational database inside it. Supabase adds authentication, HTTP access to data/functions, object storage, and hosted Edge Functions. Choosing Supabase does not outsource schema design, access policies, testing, or incident response.

Technology	Receives and returns	Provider versus Sustain responsibility
Supabase Auth	Credentials/proofs in; user/session tokens out	Provider identity service; app implements callbacks, entry state, remembering, device exchange and recovery UI.
PostgreSQL / SQL RPC	Validated mutation request in; committed result or conflict out	Database transaction machinery; app owns functions, schema, grants, ownership and migration compatibility.
Edge Functions	Authenticated HTTP requests; bounded JSON results	Hosted execution; app owns validation, secrets use, provider calls, quotas and error handling.
Storage	JPEG bytes and object paths; authorized downloads	File service; app owns privacy policies, metadata consistency, cleanup and lifecycle.
SQLite / IndexedDB	Cached vitals and queued writes	Local transaction engines; app owns reconciliation and account scoping.
AsyncStorage	Small key/value state	Ordinary persistence; app owns schema versions, draft semantics and cleanup.
SecureStore / Keychain	Protected credential material	OS-protected access; app owns binding, rotation, expiry and failure policy.
HealthKit / notifications	Permissioned reads or schedule requests	OS data/delivery systems; app owns import mapping, schedules and honest status.

An **Edge Function** is server code invoked on demand rather than an application server process you provision and maintain directly. Sustain's five entry points are `estimate-meal`, `coach-chat`, `resolve-food-barcode`, `biometric-auth`, and `delete-account`. The functions still face time, memory, concurrency, and upstream limits.

Native and web adapters have different capabilities. IndexedDB is a browser database; SQLite is an embedded database in the installed app. Browser notification and HealthKit adapters do not certify native behavior. An Expo Go session cannot substitute for the custom native build required by the HealthKit bridge.

C04, C08-C10, C15, C18, C21. R01-R04 explain managed-platform boundaries and policies. Exact deployed limits/configuration remain an operational verification task.

9. Ownership and sources of truth

A **source of truth** is the authoritative place for a particular fact. Sustain deliberately has several kinds of state rather than treating everything as one synchronized object.

Category	Example	Authority and lifecycle
Recorded fact	Consumed meal, completed workout, reading	Cloud row after confirmation; manual vitals can first exist as durable local pending facts.
Editable definition	Private food label, recipe	Owned database record. Archiving removes future selection without rewriting history.
Historical snapshot	Nutrients accepted in a meal; dated goal	Stored copy preserves the meaning at that time.
Derived result	Daily calorie total, streak, chart bucket	Recomputed from facts/rules and coverage; not an independent proof of activity.
Preference	Summary order, routine schedule, units	Layouts/reminders are account-scoped local state; setup/profile fields have cloud persistence.
Draft	Unfinished meal/workout/helper inputs	Device-local work, not a completed server event.
Operational record	Outbox item, mutation receipt, deletion token	Supports safe retry/recovery; not health activity.

Why a copy can be intentional

Suppose a saved label changes from 120 to 130 calories per serving. A historical meal stores the reviewed serving basis and consumed result, so that correction does not silently rewrite yesterday's intake. The duplicate values are intentional audit information, not merely a cache to refresh.

A **cache** is a local copy used for speed or availability. The vital cache may contain newer unsent local changes, so refresh cannot simply replace it with whatever the server returns. An **outbox** records work awaiting delivery; cache and outbox must agree about whether a local change exists.

Cross-device expectations

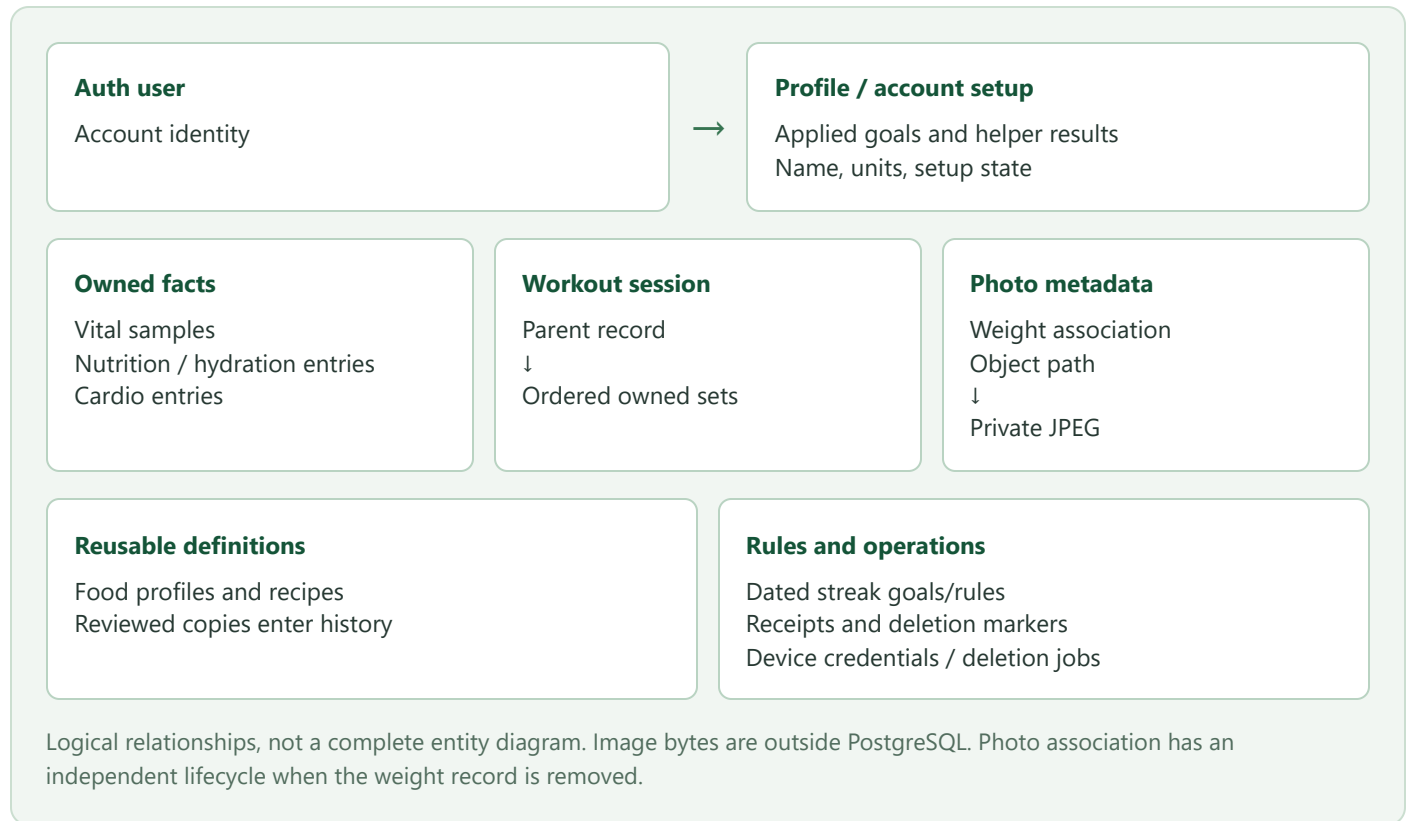
Confirmed health history, goals, food definitions, recipes, photo metadata and supported account/setup data travel through Supabase. Summary layout, reminders, reminder completions, drafts, protected device credentials and HealthKit anchors remain device-specific. A second phone can see saved meals without inheriting your reminder schedule or unfinished workout.

Cloud persistence is not instantaneous push synchronization. Current code explicitly reads and refreshes data; a Realtime subscription architecture is not part of this design. A local account key prevents ordinary accidental mixing, but it is not cryptographic encryption of the stored value.

C04, C07-C20. See page 38 for an exact offline matrix and page 35 for export coverage gaps.

10. Relational model and historical meaning

A **relational database** represents data as tables with keys connecting rows. A primary key identifies a row. A foreign key checks that a referenced parent exists. A same-owner relationship additionally checks that a child cannot attach to another account's parent.



Different time fields answer different questions

`occurred_at` answers when the activity happened. Creation/update timestamps answer when the app recorded or changed it. A row version detects competing edits. A goal's effective day identifies which target applies. Source/external IDs preserve where a reading came from. None can safely substitute for the others.

Flexible JSON fields store reviewed food/recipe structures and calculator provenance without making every optional input a new table. The cost is that validation must inspect those structures; relational constraints do not automatically understand every nested business rule.

Profiles use the account ID as `id`; most private facts use `user_id`. Shared barcode catalog data is reference data, not another user's meal history. Administrative credentials, receipts and quota records have restricted access. These exceptions matter when auditing ownership rather than mechanically expecting identical columns everywhere.

C09, C11-C14, C18, C20. Source includes 36 ordered migrations; schema presence alone does not establish deployment.

11. Constraints, indexes, and query contracts

An input screen can be bypassed. A **constraint** is a database rule that still applies when a request reaches storage another way. Sustain uses required fields, numeric bounds, foreign keys, uniqueness and ownership rules alongside client validation. Workout writes validate sets as a group before accepting the transaction.

An **index** is an auxiliary lookup structure that helps find relevant rows without scanning every account's history. Existing examples include (`user_id`, `occurred_at`) for vitals, nutrition and fluids; account/completion time for workouts; account/correlation for BP; source/external-kind uniqueness for imported readings; and account/change sequence for vital synchronization.

Indexes are not free: they consume disk and make writes maintain additional structures. Their existence does not prove that every query uses an efficient plan. A production investigation should inspect realistic query plans and returned row counts before adding more.

Pagination: retrieving history in pieces

One response can have a row cap. **Keyset pagination** asks for IDs after the last ID already seen. `collectPages` continues until an empty page and rejects nonadvancing cursors. This avoids skipping records because another device deleted a row before an offset. Summary's `allRows` uses a similar ID-based approach.

This is not a frozen multi-table snapshot. A new randomly ordered ID behind the cursor may appear only on the next refresh. It also does not bound total memory: collecting every page still builds one large array.

The export implementation has a different contract: it requests 1,000-row offset pages and stops if fewer than 1,000 arrive. If a server enforces a smaller cap, a nonfinal response is mistaken for the end. That is a confirmed conditional correctness defect, not a measured production incident.

Deeper dive: authorization belongs below the UI

Row-level security (RLS) attaches account-access rules to table operations. Critical mutations additionally use guarded SQL functions and restricted grants. A function marked `SECURITY DEFINER` runs with elevated authority, so it needs explicit account/session checks and a controlled search path; otherwise the convenience can become a bypass. [R03]

C08, C09, C19. Database isolation/locking mechanisms: R04. Query efficiency and concurrent multi-connection behavior remain unmeasured.

12. Accounts, sessions, and sign-in

Authentication establishes which account is acting. **Authorization** determines what that account may do. Signing in successfully must not grant access to another account's readings; that second decision belongs to ownership policies and guarded functions.

Supabase Auth handles email/password signup and sign-in. The app validates fields, handles confirmation and password-recovery callbacks, and provides check-email/reset screens. Signup has an eight-character client password minimum; sign-in does not incorrectly apply signup rules to an existing password. Actual email delivery, password policy and allowed redirects also depend on hosted configuration.

A **session** is the application's current authenticated relationship with the service. Its access token is a signed credential used on requests; a refresh token supports renewing access. Sustain explicitly sets `persistSession: false` and `autoRefreshToken: true`. Tokens remain process-local. Legacy persisted sessions are removed.

What the user experiences

Remember me saves account identity for convenience. It does not keep an ordinary reusable password or restore a saved Supabase session after process termination. A fresh launch requires online password authentication or the separate Face ID credential exchange. An existing process can retain its session while backgrounded; backgrounding and force-quitting are different events.

Auth callbacks are parsed and checked rather than treating any incoming URL as a trusted redirect. Signup awaiting email confirmation is not a completed login transition. Failed critical security/setup reads produce a recoverable state rather than pretending the account is new or ready.

The current UI exposes email authentication. The `/phone` destination redirects to email sign-in. Supporting phone models and legacy contact pathways do not establish a current SMS or multifactor-enrollment product.

Failure and sign-out boundaries

Local sign-out ends this session and prepares reminder cleanup without automatically revoking the remembered Face ID credential. The security screen also offers global sign-out with credential revocation. Async work checks that the active account/session still matches before applying results; a late response must not affect a newly signed-in user.

Network errors alone are not evidence of session revocation. Confirmed rejection is handled differently so offline drafts and pending changes are not erased merely because a request could not reach the server.

C05, C06. Verified synthetic auth/onboarding/entry regressions; live email links and process/Keychain behavior remain device and hosted checks.

13. Face ID: two different operations

Face ID can unlock a privacy cover over an existing session, or release a protected device credential that obtains a new session. Those operations share a biometric interaction but have different security effects.

Enrollment and fresh-session exchange

1. The signed-in user enables Face ID and proves the current password. The server independently authenticates that password and verifies that it belongs to the bearer account.
2. A random 256-bit secret is bound to the account and installation. The server stores a SHA-256 hash, not the raw credential. The installation marker is random software identity, not hardware attestation.
3. The client stores the secret through SecureStore with biometric access, device-only/passcode-required Keychain settings. A public account marker contains identity information, not the secret.
4. At sign-in, a successful protected read releases the secret. `biometric-auth` checks binding, hash, revocation and expiry, consumes/rotates it, and returns replacement credential material plus a fresh Auth session.
5. The client persists the replacement before accepting the session. Rotation retains the original 90-day expiry; it does not extend it indefinitely.

Why failures are not all equivalent

A cancelled prompt leaves the user at sign-in. A transient gateway/session error should not automatically delete enrollment. Explicit invalid-credential or incompatible-protocol responses lead to password recovery/reenrollment. Credential parsing normalizes missing, empty and null phone values consistently for email-only accounts while retaining real-phone binding checks.

The client removes the consumed protected item before creating its replacement. This avoids another prompt associated with updating the existing protected item, but creates a recovery boundary: a crash during rotation persistence or lost exchange response can require password sign-in and reenrollment. The server cannot safely keep accepting a consumed secret indefinitely.

A defensible but expensive choice

This custom flow avoids persisted ordinary Auth sessions and supports per-device revocation. It also adds security-sensitive protocol, storage and network-failure complexity. A standard supported refresh-token or passkey-based approach would be a credible alternative to evaluate before broad distribution. No claim is made that this bespoke mechanism is universally safer.

C06, C21, migration 202609180003. Unit checks cover binding, rotation, replay, expiry and persistence failures; historical hosted records and physical Face ID evidence have different scopes.

14. Privacy and access control

The app-lock behavior is intentionally tied to Face ID enrollment. There is no separate universal password-lock setting. When enabled, background/resume locking hides private content and accessibility descendants while retaining the mounted navigator and most draft state. Quick-log overlays are dismissed at the lock boundary; ordinary tab state need not be rebuilt.

This is **interface privacy**, not encryption of all data. SQLite, IndexedDB, AsyncStorage, cached images and pending drafts need their own threat-model review. The generic secure-store adapter can fall back to ordinary storage on web or native failure; the dedicated protected biometric secret path is separate. A sensitive deletion recovery capability should not be assumed protected merely because a wrapper is named secure.

Server-side protection

Private reads use authenticated account policies. Mutations check account identity and active session, validate allowed tables/fields, and enforce ownership in SQL. Same-owner foreign keys prevent attaching a valid user-owned child to someone else's parent. Deletion guards block covered access while account cleanup is underway.

Public/publishable Supabase configuration is expected in the app. Administrative service-role keys and AI provider keys are not. Server handlers using privileged clients must derive identity from verified credentials and restrict their work; possession of a request's `userId` is not proof.

Revocation is bounded

Revoking a device credential blocks future exchanges. It does not retroactively erase every access token or signed photo URL already issued. Critical writes and incremental vital reads check the current session record. Ordinary permitted reads may remain usable until token expiry; signed image URLs remain capabilities until expiry. Global sign-out is therefore not a universal instantaneous recall mechanism.

What this implementation does not establish

It does not prove end-to-end encryption, universal immediate revocation, complete secure erasure of another offline device, regulatory compliance, or a fully audited custom authentication protocol. RLS is necessary but does not replace storage policies, restrictive grants, input validation, secret management, and tests that deliberately act as another account.

A practical public-release security review should trace each sensitive item from collection through local persistence, upload, provider use, export, deletion and backup retention. That is more informative than simply counting login controls.

C05, C06, C08, C09, C18, C20. R03 supports RLS semantics; release disclosure obligations are discussed with official sources on page 49.

15. App entry and onboarding

Sustain's symbol and wordmark provide continuity between splash, authentication and the authenticated destination. The implementation uses crops of the approved artwork and its alpha mask, with light/dark backgrounds. The animation is a presentation of entry state, not a substitute for security checks.

An explicit entry state machine

A **state machine** defines allowed stages instead of relying on unrelated booleans. The entry provider tracks idle, authenticating, resolving, entering and error phases. Numbered attempts reject stale callbacks; synchronous submission guards reject duplicate taps.

Manual authentication uses a 2,100 ms repeating contained mint highlight while pending. Successful manual entry has a 320 ms fill/pulse and 200 ms fade. Biometric/unlock success is shorter, 180 ms plus 140 ms; native biometric prompting remains visually quiet. A valid attempt can trigger one light iPhone haptic. Reduce Motion skips motion and haptics.

An opaque cover blocks interaction and accessibility while critical state resolves. It waits for the intended path, matching session identity, security readiness, foreground state, privacy unlock and valid setup state. It does not wait for all health history, photos or imports. A 12-second recovery state offers retry/sign-out instead of an endless cover. Account-deletion recovery takes precedence.

First use versus first authenticated setup

Welcome is a first-use device introduction. Account creation/sign-in comes before account-scoped onboarding. The setup record retains versioned name, units, progress and completion/dismissal. The steps cover name, goals, fluids, convenience and summary; established accounts do not replay setup merely because they relaunch.

Onboarding reuses the same persistent calorie/fluid GoalHelper as Profile. It distinguishes the applied target from the last estimate and preserves unrelated unfinished fields when a helper result is applied. Skipping optional setup does not manufacture health activity.

Keyboard handling uses one native inset-aware scrolling owner, with actions scrolling as part of the form and an iOS Done accessory where appropriate. Browser short-viewport checks support layout confidence but do not reproduce native keyboard mechanics. Standalone splash continuity, real haptics and prompt timing require a rebuilt iPhone client.

C05, C07, C15; ADR-010. Synthetic entry and onboarding tests passed this review; native build configuration alone is not device validation.

16. Profile, entry dates, and navigation

Profile groups Personal Details, My Goals, Preferences, Health Data, Account & Security, Sign Out and Manage Account. Nested routes retain the landing screen's scroll position. Focused sheets handle short edits; longer destinations have their own screens. Loading, errors, unsaved-change confirmation and destructive confirmation are shared patterns.

Names are coordinated through existing profile/account setup information. Unit preferences change presentation while preserving canonical values. Appearance supports System, Light and Dark. Sync & Pending Changes and Apple Health are separate because uploading local changes and importing device data are different jobs. Reminders are reachable from Track; the old Profile route redirects to that destination.

A calendar day is not UTC midnight

Users can select an entry day for manual weight/BP, meals, fluids, lifting, cardio and photos. A shared calendar-icon field defaults to today and rejects future days. `entryTimestamp` takes the selected local calendar date and the current local clock time, then produces an ISO timestamp. The result records a chosen day, not an independently measured historical time of day.

For example, logging yesterday's dinner today uses yesterday's local date. Converting a bare `YYYY-MM-DD` through UTC midnight could shift the displayed date in another timezone; the shared helper deliberately uses local calendar construction. Durable pending operations freeze their timestamp so retrying tomorrow does not move the entry to tomorrow.

Why screens share structure

Lifting and Cardio keep Entry date, Plan workout with AI and the type switch in stable positions. Separate draft models preserve their contents when switching tabs. Fluid quick amounts fill the amount field without silently selecting Water; the user still chooses the drink classification.

Delete dialogs share consistent action layout and explicit cancellation. Reminder medication list/editor navigation stays inside one persistent full-screen modal to avoid flashing the underlying page. Shared settings rows permit text wrapping so a long label does not displace its centered status value.

These choices favor predictable interaction over hidden autosave of consequential changes. They do increase dependence on shared primitives: one change to a sheet or input can affect several features. Tests measure selected geometry and state transitions; VoiceOver, Dynamic Type and native keyboard acceptance remain separate work.

C03, C07, C15, C16; `lib/entry-date.ts` and `ui entry-date/input/sheet` components. Dates represent user-entered history, not verified occurrence.

17. Summary widgets and charts

The widget registry is the authority for available Summary content. It defines each widget's title, supported sizes, configuration schema and data requirements. The twelve types are Weight, Blood Pressure, Calories, Protein, Fluids, Calorie Calendar, Weight Trend, Blood Pressure Trend, Today's Meals, Training Summary, Quick Actions and Streaks.

The eight default widgets are the two latest-vital cards, three daily nutrition/fluid cards, calorie calendar and two trends. The optional widgets expose meals, training, shortcuts and goal streaks. There is no selectable personal-record widget, although training-analysis code remains.

Editing without changing the underlying records

Layouts are validated, account-scoped local preferences. Edit Summary supports rearranging, adding, removing and choosing Small/Large sizes (Large is stored as wide). Streaks and Quick Actions also have direct edit icons. Preview uses real widget renderers; configuration is kept separate so selecting a streak does not change the height above the options and move the page.

Validation rejects unsupported sizes, duplicate IDs/configurations and invalid action/habit selections. A Small Streaks widget contains one habit. Quick Actions uses two items when small and four to six when wide. Save commits the layout; cancel restores it. Saving newly selected habits initializes only absent tracking rules; a preview is not authorization to initialize history.

Data and presentation

Summary loads relevant source domains and records coverage/errors. Daily totals derive from saved entries; manual/HealthKit readings are combined for applicable vital displays. Optional historical streaks can require substantial history. Missing coverage should be visible rather than displayed as confidently zero.

Each trend owns its selected range. SVG curve construction uses slope limiting to avoid smoothing that invents an extreme beyond observed values. The calorie calendar aggregates per local day and fills rings from empty to the loaded calorie/goal fraction over 650 ms, or immediately under Reduce Motion. Goal loading precedes initial totals presentation so a temporary missing denominator does not produce a misleading full ring.

Tradeoff: local customization is inexpensive and responsive but does not transfer to a second phone. Recomputing historical cards avoids stale stored counters but can grow in read volume and client memory. These are separate scaling concerns from the number of accounts.

C10, C11. Tests cover layout migration, retirement filtering, current-week rendering, data coverage and native Text-host rules.

18. Goal streaks and historical rules

A streak is a **derived result**: the engine evaluates recorded evidence against rules and dated targets. It is not a counter incremented whenever the app opens. That distinction makes backdated entries, edits and deletions meaningful.

Offered streak	Qualifying evidence
Calorie target	A nonempty saved food day satisfies the inclusive under/over target rule. Empty days do not qualify. Compatible range/tolerance rules remain readable.
Protein target	Logged protein reaches the positive target effective on that date.
Fluid target	Counted beverage volume reaches the positive dated fluid goal; alcohol contributes zero, unresolved status remains pending.
Training consistency	Chosen 1-7 distinct training days within a Monday-Sunday week. Lifting needs valid saved sets; cardio needs positive duration.

Two workouts on one date contribute one training day. Reminder completion, logging alone and a food-day confirmation are not offered streak types. Legacy rows are filtered rather than destructively deleted, preserving compatibility without advertising inactive behavior.

Periods and uncertainty

The engine uses dated rule revisions, activation dates, goal snapshots, source coverage and an injected clock. It returns met, open, not-met, unknown or not-scheduled periods, with evidence and explanations. Missing historical targets are not replaced with today's goal. Missing complete source coverage is not proof of failure.

An unfinished current period is provisional. It can extend a current run if qualified while an open unfinished period preserves the prior run. Closed failures break runs; paused/off periods are neutral. Best excludes the unfinished period, and unknown gaps prevent claiming a verified consecutive sequence.

All cards display the current Monday-Sunday initials, checks and dates. Training checks mark actual training days, not seven copies of the weekly outcome. Detail history can have different activation dates because evidence/initialization differs by habit; those dates are not simply the last success.

Historical goals versus current dashboard goals

Goal changes create prospective dated snapshots for streak evaluation. Automatic protein resolves a weight-based target and retains provenance. The live dashboard can show the current profile target while historical streaks use their dated denominator. This preserves reproducibility at the cost of explaining why two views may differ after an edit.

C11, C15, migrations 202609180001/202609220001. Pure engine tests exercise boundaries, edited totals, unknown coverage and weekly semantics.

19. Weight, BP, pulse, and history

Manual vital logging supports a selected day, weight in display units, and paired systolic/diastolic blood pressure with optional pulse. Forms validate numeric inputs and required relationships before queueing. Weight/BP goals and category labels are contextual wellness displays, not diagnostic measurements.

The model keeps each reading's ID, account, kind, value/unit, occurrence time, source, optional external/correlation ID, version and deletion state. Systolic and diastolic records share a correlation so they can be treated as one reading in the UI. Pulse can be associated without turning unrelated heart-rate history into a general import feed.

The user's save and edit experience

Manual saves update the local vital cache and outbox together, so they can be retained without immediate cloud confirmation. A pending/error status distinguishes local success from synchronization. History allows manual corrections and deletion; a deletion is represented locally by a marker and queued like other changes. Conflicts preserve the local intent for explicit recovery rather than silently overwriting it.

Source-specific history retains origins. Combined displays use a near-duplicate heuristic: equal kind/value within five minutes collapses for aggregation, with manual preference. Weight compares converted pounds within a small tolerance; other kinds use their own numeric comparison. The implementation can collapse nearby duplicates without requiring different source labels, so it is a display heuristic rather than proof of identical physical measurements.

Charts do not add new measurements

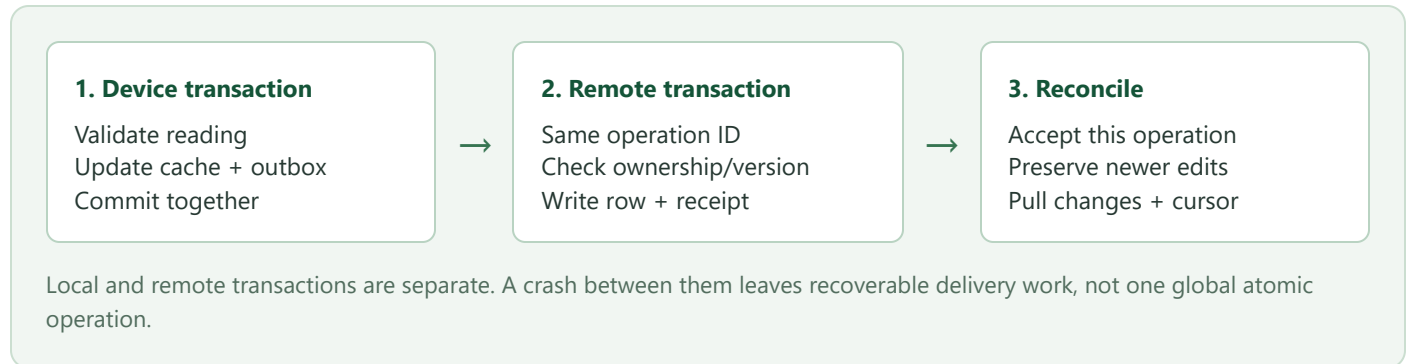
Time-range selection, bucketing and trend rendering summarize available data. Curved lines are interpolation, not additional readings. Incomplete BP pairs do not qualify as complete paired BP evidence. A category or average should be interpreted alongside dates, units and available observations.

History and graphs can differ in apparent count because the latter may deduplicate or aggregate. Deleting a source record and suppressing a visual duplicate are different operations. Preserving original source identity makes that difference auditable.

Verification: domain/unit tests cover conversions, pairing, pulse association, chart geometry and queue state. Browser tests exercise synthetic save/edit/retry paths. No hardware BP device or clinical measurement accuracy was tested in this review.

C08, C12. Readings are user/provider facts with provenance; the app does not verify that a manually entered number was actually measured.

20. Walkthrough: saving and syncing a vital



On native, SQLite stores account cache and outbox payloads in two tables inside an exclusive transaction. Browser IndexedDB uses an account transaction. These are account-sized JSON payloads, not yet normalized row-per-reading SQLite storage. Failed outbox persistence must roll back the matching cache change.

Each queued operation retains its base version and dependency chain. Sync checks the active account, prepares the frozen request, sends it and accepts only the acknowledged operation. If the user edited again while a request was in flight, the acknowledgment cannot remove the newer work. A remote conflict remains visible in Sync & Pending Changes.

Deeper dive: why the cursor matters

A **cursor** is a saved position in an ordered change stream. PostgreSQL assigns a per-user `change_seq` under a counter lock held through commit. `read_vital_changes` establishes an upper bound for a pass and returns changes through that bound. The next pass can pick up later changes.

The client commits each downloaded page and its cursor together. Otherwise a crash could advance the cursor without saving those readings, permanently skipping them. Tombstones preserve deletion knowledge; an old acknowledgment must not resurrect a record removed elsewhere.

Using only `updated_at` would make equal timestamps and commit ordering harder to reason about. An ordinary sequence allocated before commit can also have visibility-order problems. The current counter/locking design explicitly addresses the traversal boundary.

Limits

BP members commit together locally, but individual cloud operations can temporarily produce a partial remote pair until remaining retries finish. Cold-start login still needs connectivity. Full-account JSON rewrites and complete cache loads may become costly with years of history; that is a reason to measure and normalize local storage later.

C08, C09. Isolated SQL and local store tests passed; genuine simultaneous database connections and device process-kill recovery need separate testing.

21. Walkthrough: Apple Health import

HealthKit is Apple's permissioned health-data store on the device. Sustain reads it through the native bridge; it does not connect directly to a BP monitor's vendor cloud. The browser can display already synchronized Apple-origin rows but cannot perform the HealthKit read.

Import sequence

1. Connect requests read access for body mass, systolic/diastolic BP and heart rate. The app records its connection preference; an empty query is not proof that Apple granted or denied each read type.
2. Synchronize existing cloud vitals first. If that fails, the import stops with an actionable message.
3. Read weight samples and BP correlations from saved anchors, initially using a one-year lookback. An anchor is HealthKit's position for incremental additions/deletions, scoped to this device/account state.
4. Map external UUIDs into stable account/kind-specific IDs. Preserve source names, occurrence times and BP correlation identity. Queue additions and anchored deletions through the same vital mechanism.
5. Synchronize again. Only after success save the new anchors and import timestamp. If the final sync fails, queued work remains and the anchors do not falsely advance.

Pulse is an association heuristic

For a BP correlation, the reader considers up to 12 heart-rate samples within two minutes before/after its time. It prefers the same named source, then the nearest sample. This does not guarantee that the pulse came from the same monitor measurement. Weight/BP deletion streams are anchored; there is no standalone general heart-rate import/deletion stream.

The app combines imported and manual readings for vital/goal evidence automatically. There is no current weight-logging streak from which imports could be excluded. Imported records retain source identity rather than being converted into fake manual logs.

Choice and limitations

Read-only access reduces writeback loops and scope. Foreground/focus coordination is easier to reason about than a continuous background pipeline, but does not promise always-current data while suspended. The pre/post cloud sync means this path is not a fully offline ingestion engine.

No workout, sleep, steps or heart-rate-variability import is implemented. The configured HealthKit purpose text mentions workout data and should be aligned with actual reads before release. Real permission denial, revocation, source duplication, deletion and timezone behavior require an iPhone test. [R10]

C12, C13, ADR-002. Mapping/identity tests are synthetic; neither a browser pass nor native export exercises the user's Health store.

22. Lifting and cardio implementation

Lifting records a session with ordered exercises and sets, repetitions, working weight, muscle groups, optional notes and gym/location text. Location is a label, not GPS tracking. Unilateral exercises retain left/right values while representing one logical set. Explicit zero load is valid for bodyweight work; a blank field is not silently zero.

From draft to completed history

An account-local workout draft survives navigation and restart. Exercise IDs, order and set values stay associated during reordering. Drag controls and accessible move actions provide two ways to change order. The common header keeps date, AI planning and Lifting/Cardio controls stable.

Save validates the session and sends `create_workout`; editing history sends a versioned replacement. The SQL transaction updates the session and all child sets together. Invalid set data rolls the transaction back. A server-confirmed result is needed before claiming completed history; an offline draft is not a completed workout.

Cardio has its own draft and saved records: activity/name, positive duration, optional distance, date and notes. It uses the same retry/version framework. A deleted or edited entry changes history and derived training totals. The app does not infer lifting duration from set count or import a GPS route.

Deterministic history guidance

Suggestions and remembered exercise performance come from the signed-in account. `getExerciseGuidance` uses a 90-day lookback with a 100-row cap. `buildExerciseGuidance` selects the strongest supplied session and flags progression only when at least two working sets at the highest load each reached 12 repetitions.

The rule suggests another 2.5 lb at loads up to 50 lb, otherwise 5 lb. This is an explicit heuristic, not a learned model or universally appropriate prescription. A bounded lookback can omit relevant history; equipment differences and ambiguous exercise names remain limitations.

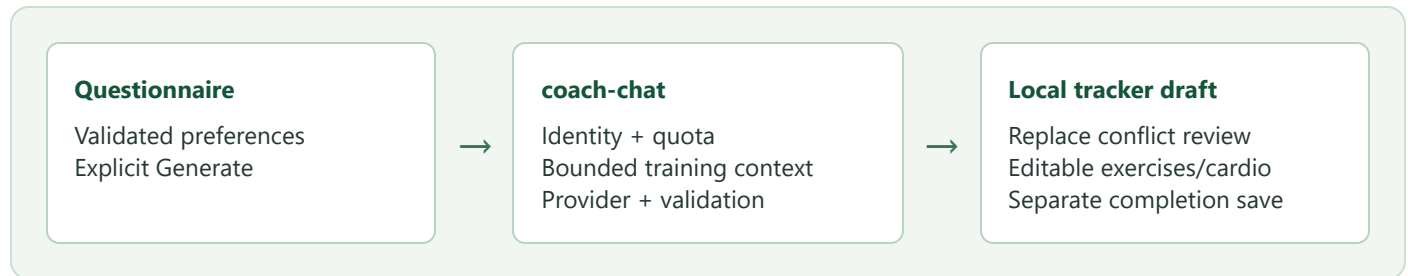
Full history paths use pagination; bounded suggestion queries serve a different purpose. Training Summary counts valid sets, completed sessions and distinct local days. Retained personal-record analysis separates side modes and excludes first baselines/ties, but is not offered as a Summary widget.

Tests: pure progression/catalog/draft/history checks, transactional SQL assertions, browser planner/draft flows. Exercise suitability and actual training outcomes are outside those tests.

C14, C09, C10. Detailed sets support reproducible history at the cost of manual-entry effort and more complex editing.

23. Walkthrough: AI workout planning

Plan workout with AI opens a three-step questionnaire for **one lifting, cardio or mixed session**. Inputs include goals/styles, experience, available time, equipment, focus, limitations and readiness, with training-history consent. Selections are validated arrays; no-equipment is exclusive. A weekly-frequency storage field remains for compatibility but is not a current questionnaire requirement.



The server builds training-only context for these requests, including recent sessions/sets/cardio and deterministic 7/28-day primary-muscle-group summaries. Broader legacy Coach permissions do not expand the current planner's context to nutrition, vitals or photos. Tools are read-only and restricted to training for this path.

The model proposes structured actions. Shared guards check requested sections, exercise shape, effort/rest/technique and a conservative combined time budget. Only a valid `next_workout` plan can populate the affected drafts. Existing content requires an explicit replacement decision. Repetitions in reserve, rest, technique and notes persist in the draft; normal tracker Save still records actual completion.

Cost and failure behavior

New generation uses the deep tier: three requests per user per UTC day and a 4,000-token output cap. Retained standard routing allows 30/day and 1,400 output tokens. Code defaults name Terra for deep and Luna for standard, with server-secret overrides; this review did not inspect live model settings. Input budgets, bounded tool rounds and timeouts constrain the request. There are no paid automatic retries.

Cancellation stops waiting locally but cannot guarantee an already-started external request is unbilled. Coach can refund app quota for certain failures; that is not a provider refund. A rejected/malformed plan remains an error, not a fabricated workout.

There is no current chat composer or conversation-history selector. Legacy threads/actions and compatible server behavior remain stored. Schema validity does not establish plan quality; representative evaluation of safety, feasibility, time realism and history use is still needed.

C14, C21, ADR-005 amendments. Current routes and handler guards were inspected; no paid generation or live model-quality evaluation ran.

24. Meals and reusable food labels

Food logging builds a local meal draft with breakfast, lunch, dinner or snack selection. An item can originate from manual entry, a private label, recent history, a barcode, a recipe, or an accepted AI estimate. These routes converge on the same portion model rather than implementing separate nutrition arithmetic.

A **serving basis** defines nutrients for a known reference amount. A consumed amount says how much was eaten. Calories and protein are required; carbohydrate, fat, fiber, sugar and sodium can remain unknown. A missing optional nutrient is not silently interpreted as zero when presenting a complete total.

Example: what the calculator actually does

A label contains 120 kcal and 5 g protein per 30 g. A consumed amount of 45 g is 1.5 servings, giving 180 kcal and 7.5 g protein. Supported portions include servings, household counts, mass and liquid units when the basis supplies the conversion. Converting grams to milliliters requires information; the app does not assume every food has water's density.

Countable foods preserve a household quantity/unit so an entry can remain "8 dumplings" instead of an unexplained decimal serving. Numerical bounds and schemas reject invalid basis or consumed amounts before persistence. These are data-quality bounds, not proof that the label itself is accurate.

Definitions versus consumed facts

Private food profiles are reusable definitions with identity/provenance information. Normalized content, brand and source/barcode identity reduce accidental merging. Name alone is insufficient to identify a food. An account-owned corrected label can take precedence for future use without changing the shared provider catalog.

Completed nutrition entries retain reviewed basis, portion, nutrient snapshot and provenance. Editing a label does not rewrite previous meals. Editing a historical amount recalculates from that entry's basis and uses a versioned mutation. Archiving removes a label from selection while retaining history. Deleting a consumed row changes totals and derived streaks.

Save meal freezes retry identity and sends the completed rows through the mutation protocol. Label saving is a separate choice/workflow and should not be described as one universal transaction with every draft edit. A lost response remains pending until confirmed; drafts survive, but completed offline meal acceptance is not implemented.

C14, C08, C09. Nutrition model, portions, unknown nutrients, draft restoration, save/retry and edited totals have automated coverage.

25. Recipes and barcode lookup

Recipes preserve a reviewed composition

A recipe stores a private name/description, declared serving yield and ingredient snapshots assembled from a recipe draft. The app totals ingredients and divides by yield. A 1,200-kcal recipe declared as four servings is 300 kcal per serving; half the whole batch is 600 kcal.

If any ingredient omits an optional nutrient, that recipe nutrient remains unknown. Recipes cannot contain other recipes, avoiding circular/nested composition. Later label edits do not silently change saved recipes. Archiving preserves historical references; modifying a preparation requires an explicit new/replacement definition rather than an invisible cascade.

Logging a recipe uses the existing portion calculator and saves one consumed snapshot with recipe identity. Decimal/fractional servings or a fraction of the whole recipe are supported. Reproducibility costs duplicated snapshot data, but keeps yesterday's meal stable.

Barcode request path

Expo Camera or manual digits supply a UPC/EAN code and type. The client can resolve the user's own label first. The authenticated `resolve-food-barcode` server function normalizes/checks the identifier, consults the shared lookup/product cache, then queries Open Food Facts when needed.

The provider response is normalized into Sustain's serving/nutrient model. The implementation calls a versioned product endpoint with a required identifying User-Agent and an eight-second timeout. Positive results have a 30-day freshness period; not-found results have a 24-hour negative cache. A stale usable product can be a fallback during upstream failure.

A cache is a reusable copy that avoids repeating an external request; a negative cache remembers a recent absence. Neither proves that a package label is correct or current. The user reviews ambiguous serving information and can create a private correction. Invalid/missing products leave manual entry available.

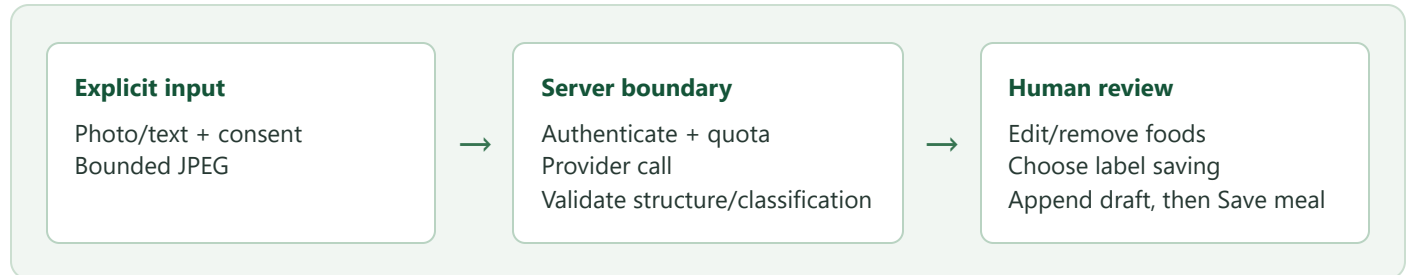
Public-use implications

Open Food Facts documents rate limits and attribution/license obligations. Because Sustain proxies lookups through its server, traffic can share upstream IP limits; account quotas alone would not solve that. Add measured cache-hit/miss and burst behavior before wider use. USDA search is planned, not an implemented fallback. [R14]

C14, C21, ADR-006. Recipe arithmetic and barcode/provider normalization tests passed; no live product lookup or provider coverage study ran.

26. Walkthrough: an AI meal estimate

The AI feature helps turn a supplied photo and/or description into reviewable foods. It does not read the user's entire health history or automatically mark a meal consumed.



Images are re-encoded as JPEG, stripping retained EXIF metadata and reducing dimensions to at most 1,440 pixels. The request requires explicit consent and text or image input. Code bounds include 4,000 description characters, approximately 2 MiB image input, up to 15 components, 20 requests per account per UTC day, a 60-second provider timeout and 3,500 output tokens.

The server derives user identity from verified authentication, consumes an atomic quota and calls OpenAI Responses. The source default is `gpt-5.4-mini`, overridable through server configuration. The actual deployed model, funding and access were not checked. `store: false` is a request setting, not a guarantee of zero provider retention.

Structured output separates one-serving basis, consumed amount, nutrients, confidence and assumptions. Server and client reject invalid shapes. Non-food or unclear classifications discard contradictory proposed foods. Name normalization and draft checks reduce duplicate components when a photo and description depict the same meal. Cooking-fat assumptions remain visible for user correction.

The user edits/removes components and chooses reusable-label storage independently per item. Accepted foods enter the existing local meal draft; normal Save meal records the immutable consumed snapshots. There is no model mutation tool capable of bypassing that flow.

Failures retain the input for review/retry and create no placeholder zero-calorie foods. There are no paid automatic retries. Quota can be consumed even if the provider fails; cancelling locally cannot recall a request already underway. Raw input/provider bodies are excluded from application logs, but temporary device image files can exist.

Quality boundary: schema checks establish structure, not nutritional accuracy. Representative weighed meals, hidden oils, mixed dishes and non-food inputs still need an evaluation set before broader rollout.

C14, C21, ADR-004. Fresh tests use provider stubs, not paid estimates. Manual logging remains usable independently of AI availability.

27. Fluids: recorded volume versus credit

Daily Fluids records a selected date, drink category/name, alcohol classification and consumed amount in milliliters, fluid ounces or cups. Quick fluid amounts only fill the amount; they leave drink selection to the user. Common and recent drinks assist entry, while a category popup exposes other choices.

The calculation converts volume to canonical milliliters using explicit US unit factors and rounds stored input consistently. The schema rejects nonpositive values, excessive volume and category/alcohol contradictions. The current screen does not expose full saved-drink management, although retained records/backend support remain.

A policy, not a physiological model

Classification/policy	Stored volume	Daily goal contribution
Nonalcoholic beverage	Actual entered liquid volume	Full volume
Alcoholic beverage	Actual entered liquid volume	Zero
Unknown classification	Actual entered liquid volume	Pending, not guessed
Legacy volume policy	Historical entered volume	Preserved historical volume

fluidTotals reports consumed, counted, pending and alcoholic milliliters separately. It does not estimate absorption, dehydration or net water retention. Coffee, tea, soda and shakes receive beverage-volume credit rather than speculative percentages. The app does not invent calories from a beverage category; food nutrition is logged separately.

Each new entry stores category, alcohol status and policy version. Database constraints enforce valid combinations and a generated contribution column supports consistent storage. Shared pure logic serves the app and compatible Coach context. Explicit reclassification changes only the chosen record; changing a reusable definition does not retroactively rewrite old logs.

Save, edit, and failure behavior

Hydration entries use durable pending mutation identity and require server confirmation. History edits/deletes use versions and recompute relevant totals/streaks. A failed or uncertain save remains recoverable, but this is not the vital outbox's confirmed local offline-write contract.

The design makes policy visible and reproducible. Its limitation is equally explicit: reaching a beverage goal is not proof of an individual's hydration status, and unknown classification can leave some recorded intake outside credited progress until reviewed.

C15, C08, ADR-007; shared hydration functions and migration 202609170002. Policy/legacy/unknown arithmetic and browser flows have coverage.

28. Goals and persistent helper state

Applied goals, calculation inputs, last estimates and measurement history answer different questions. Sustain keeps them distinct so exploring a calculator does not silently change the target used by the app.

Value	Meaning
Applied calorie/fluid goal	The number the user explicitly accepted or entered manually.
Latest calculation	A saved estimate with inputs, method, assumptions and calculation time.
Helper draft	Unfinished local field edits, potentially invalid or incomplete.
Calculation weight	The weight used in that estimate.
Latest reading	Current manual/Apple Health evidence; not an automatic rewrite of an estimate.
Target weight/date	A planning objective, not a measured result or guaranteed outcome.

Reopening, calculating, and applying

The shared GoalHelper restores local drafts and stored profile calculation JSON. Successful estimates persist under a versioned `latestCalculation` structure before acceptance. Recalculate uses the current date and explicit inputs; Use This Goal changes the numeric target and applied provenance. A manual target can preserve the last estimate for reference.

Per-account drafts retain exploration across closing, tabs and login. Display-unit switching preserves canonical numbers to avoid repeated rounding drift. Loading errors and invalid inputs keep the previous result separate rather than making it look like a newly calculated value. A failed metadata save is recoverable through the same pending-operation mechanism.

Profile and onboarding reuse this logic. Accepting one helper changes only its intended fields; it does not replace unrelated protein or target-weight drafts. The latest imported weight can be deliberately adopted for a calculation, but an import does not continuously change a saved calorie/fluid goal.

Cloud writes and historical targets

Goal writes compare only changed fields against their captured baseline. This allows independent edits, such as calories on one phone and fluids on another, to merge while rejecting competing changes to the same field. Profile goals and calculator metadata persist in the cloud; unfinished helper inputs remain device-local.

Streak goal history uses effective-day snapshots instead of applying today's goal to all previous days. Goal editing is therefore both a current preference change and a historical-rule operation, which justifies more care than writing a single number to local storage.

C15, C07, C08, C09. Pure calculation/draft tests pass; date-frozen browser limitations are reported in the verification chapter.

29. Calorie helper: exact implemented math

The helper estimates resting energy with the Mifflin-St Jeor equation: $10 \times \text{kg} + 6.25 \times \text{cm} - 5 \times \text{age} + \text{offset}$, where the implemented male offset is +5 and female offset is -161. The UI explains the equation category; it is not a complete representation of gender identity or individual physiology. [R15]

Maintenance is resting energy multiplied by the selected activity factor: **1.2, 1.375, 1.465, 1.55, 1.725 or 1.9**. These particular factors are application inputs documented in the goal evidence notes, not individually measured expenditure. Workout calories are not added again on top of that factor.

Worked example

For age 30, 80 kg, 180 cm and the male equation, resting energy is $800 + 1125 - 150 + 5 = 1780$ kcal/day. With factor 1.55, estimated maintenance is 2,759 kcal/day. An illustrative 1 lb/week loss scenario subtracts 500 kcal/day, producing 2,259 kcal/day. This explains the arithmetic; it is not a recommendation for a particular reader.

The primary loss plan uses a target calendar date. Calendar ordinals measure the day interval without daylight-saving drift. Supported duration is 7-3,640 days, with required loss of 0.1-2 lb/week. The app converts that rate to a daily deficit using $\text{rate} \times 500$; it rejects invalid dates, rates or unsupported results instead of silently substituting another plan.

Comparison options expose 0.5, 1, 1.5 and 2 lb/week. Maintenance uses the estimate directly; gain options are the app's +5% or +10% convention. The calculation is a short-term planning heuristic, not a dynamic body-weight model accounting for metabolic adaptation.

Validation and limitations

Code restricts the calculator to adults 19-120, heights 100-250 cm and weights 30-500 kg, with additional body-mass-index and scenario checks in helper logic. Generated targets below 1,000 or above 20,000 kcal are unavailable. Those software bounds do not establish an individually adequate intake. Unsupported medical/pregnancy/breastfeeding contexts are outside the helper's intended scope.

Unrounded calculations are retained internally; display/save use whole calories. A new estimate is not an applied goal until accepted. Manual targets and observed progress remain necessary ways to adjust a population-based estimate.

C15, GOAL_HELPER_EVIDENCE.md. Formula source R15; all exact activity factors, bounds, date/rate rules and gain conventions are verified in code, not presented as universal medical guidance.

30. Fluid, protein, weight, and BP goals

Fluid suggestion and custom mode

Suggested fluids use adult beverage references of 3,000 mL for the male category and 2,200 mL for the female category. Total-water reference values of 3.7/2.7 L include food and are not substituted as drink-only goals. These are population references, not individual minimums. [R16]

The app adds a transparent usual-activity allowance: mostly sitting +0 mL, lightly active +250, moderately active +500, very active +750. Those increments are product planning assumptions, not measured sweat loss. The helper does not ask for workout minutes, climate or a weight multiplier and does not claim that the extra numbers are clinically validated.

Custom mode accepts a positive goal up to 20,000 mL without requiring sex/activity inputs. It preserves canonical milliliters across unit changes and adds nothing automatically. The saved goal stays fixed until the user applies another value. The separate snapshot preserves how the last suggestion was obtained.

Protein and historical weight evidence

An explicit daily protein goal takes precedence. Without one, the display computes approximately **0.7 g per pound of the latest available weight**, rounded to a whole gram. Manual and imported weight both participate. A newer synchronized reading refreshes the automatic display; it does not change the user's explicit protein target.

Dated streak snapshots resolve automatic protein from available weight for that date and preserve source provenance. This avoids silently using a new body weight for every historical day. No weight means no invented automatic denominator. The 0.7 rule is an implemented heuristic, not a complete individualized nutrition prescription.

Weight and blood-pressure targets

Weight target is stored independently of readings, with normalized canonical pounds. BP targets are separate systolic and diastolic values. Repository schemas enforce positive bounded values; they do not establish that a chosen target is appropriate for a person. Editing goals does not create a reading, change a recorded measurement, or award a logging streak.

These goal types illustrate why source of truth matters: an objective is a preference, a reading is a fact, a calculator output is an estimate, and a streak is derived evidence. Combining them into one field would make historical interpretation and retries much less clear.

C15, C12, C11. Shared calculator and goal-display tests cover formulas, conversions and saved-versus-automatic behavior.

31. Reminder setup and completion

Reminders lives in Track. Meals, Fluids, Blood Pressure, Weight and Workout each have their own enabled state, weekday choices and time slots. Medication / Supplements is a sixth consistent row with a centered toggle and an editor containing multiple named items. There is no global routine-enable switch or quiet-hours setting.

Meals has breakfast/lunch/dinner slots; BP has morning/evening; Weight and Workout have their corresponding slot; Fluids supports editable slots. Duplicated times within a category are rejected; overlap across categories can require an explicit choice or notice. Workout days start unselected so a schedule is not assumed.

Medication/supplement items support individual enabled state, selected days, up to six times per day, and every-2-to-365-days recurrence with a start date. Pausing the medication group masks effective scheduling without overwriting each item's choices or completion history. Named custom reminders keep name and schedule controls; new custom setup does not offer dedicated BP/weight/medication types again.

Editing is local, scheduling is a second action

Validated definitions are stored in account-scoped local persistence. Routine data is version 2 while retaining the existing storage-key namespace for migration. Legacy master-off choices are migrated without accidentally enabling categories; missing medication group state preserves existing enabled delivery.

Editors use full-screen presentation. Medication list and item setup share a persistent modal; delete confirmation keeps unsaved editing state underneath and hides covered controls from accessibility. Cancel leaves the editor intact. Failures remain actionable rather than dismissing the user back to the main page.

Custom reminder completion is explicitly recorded per local day/slot. A notification tap is not completion and not proof that medication was taken. Routine schedules do not enter completion denominators or write goal-streak credit.

Ownership boundary

Definitions, completion history and OS request IDs are device-local. A cloud account does not synchronize them to another installation. Deleting a definition cancels its owned requests, while cleanup handles stored identifiers. The absence of a settings message about local storage does not change that implementation boundary.

C16, C17, ADR-009 amendments. Fresh browser cases cover row/toggle/editor/delete behavior; actual operating-system delivery remains unverified here.

32. How notification scheduling works

A **local notification** is scheduled with the phone's operating system. It can fire while the app's JavaScript is not running. A **remote push notification** begins on a server and travels through a push service. Sustain implements the former; it has no cloud reminder scheduler.

Schedule	Native request strategy	If the app is not reopened
Stable daily/all-week	Repeating calendar time	OS can continue repetition.
Selected weekdays	Calendar repeat per weekday/time	OS can continue repetition.
Workout routine	Dated requests over 28 local days	Stops after the saved window.
One-time custom	Single dated request	Only that occurrence exists.
Interval/future-start/early-completion exception	Next 14 valid dated occurrences	Requires foreground replenishment.

The serialized scheduling service computes desired requests with stable account/source/slot identifiers and fingerprints. It compares them to the OS pending list, adds/replaces and verifies new requests, then removes obsolete ones. This avoids a cancel-everything gap and makes partial retries recoverable.

The app budgets 60 pending requests and reserves four transition slots. This is a conservative application policy, not a verified universal current iOS limit. Other-account custom requests count toward capacity. A capacity error is surfaced rather than silently truncating schedules. Explicit off/delete cancels its requests even if another enabled group cannot be rebuilt.

Time, permission, and partial failure

Permission denial retains choices and offers system settings; repeated prompts are avoided when the OS cannot ask again. Saving definitions does not itself prove OS scheduling succeeded. Add-before-remove verification can still leave an older or partially updated schedule during failure, so status must not claim a completed reconciliation.

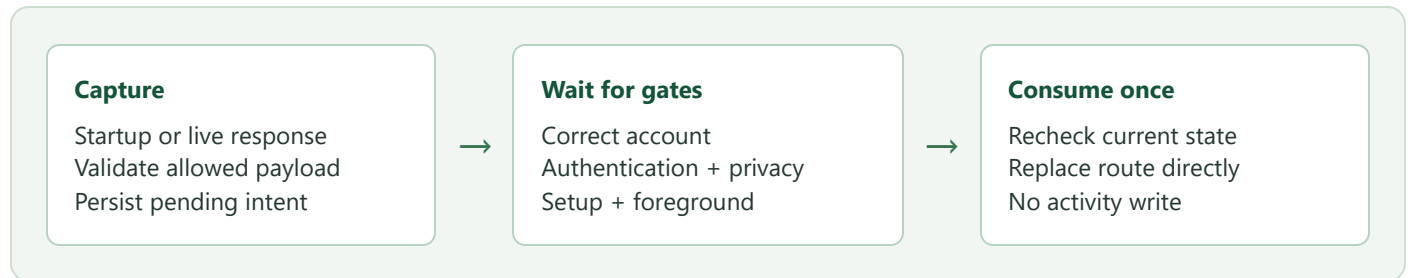
Calendar components follow intended device wall-clock time. Dated schedules advance local dates rather than adding fixed 24-hour intervals. Workout spring-forward nonexistent times are skipped; repeated/folded times and timezone changes while suspended still depend on iOS behavior. Rest Today cancels today's remaining workout prompts and stores a local preference; it creates no workout or streak credit.

Automatic suppression after saving food, fluids or a reading is not implemented. Stable reminders may still arrive after logging. A finite rolling window is not indefinite background delivery.

C17, ADR-009; R11 explains OS scheduling APIs. Deterministic scheduling tests pass, but permission/retention/DST delivery need physical-device evidence.

33. Walkthrough: a reminder while signed out

A notification is a request to open a destination, not authorization to access private data or write a completed log.



An **intent** is a small structured description of what to open. Version-2 payloads contain account identity and a known routine/custom category/slot. They do not contain arbitrary executable URLs or tokens. Legacy IDs resolve only against uniquely owned saved definitions.

The startup response is captured before choosing the entry destination. The live listener feeds the same serialized pathway. Response identity, delivery timestamp and action are deduplicated; the seen list is bounded to 100 keys. A pending intent expires after 30 minutes, and a newer valid tap can replace it.

If signed out, the user signs in first. Failure does not discard a still-valid intent. Required onboarding completes before routing; completed accounts can enter the logger directly without first mounting Summary. Privacy lock and foreground readiness also gate consumption. A final identity/lock check after asynchronous work prevents a late response from routing into the wrong account.

Meal slots can preselect breakfast/lunch/dinner only when they will not overwrite an existing draft. The old notification's date does not backdate the new log. Opening a medication reminder does not mark it taken. Draft, date and pending-save state remain controlled by the destination.

Lifecycle differences worth disclosing

Explicit sign-out cancels outgoing routine requests and pending navigation while retaining preferences. Existing custom schedules can continue; ordinary logout does not mean all custom prompts disappear. Automatic session expiry preserves intent/settings for recovery. Account deletion performs broader owned routine/custom/delivered-notification cleanup.

This extra state machine is justified by cold start, authentication, account switching and setup races. A direct router.push inside a callback would be simpler but would skip those independent boundaries.

C05, C16, C17. Fresh synthetic cases verify direct destination, failed login, incomplete setup, deduplication and draft protection; real taps still need an iPhone.

34. Walkthrough: private progress photos

Photos are linked to weight entries but stored as two resources: a PostgreSQL metadata row and JPEG bytes in a private Storage bucket. New uploads require an active owned weight reading and allow at most three attachments per entry. The quota is not three photos per day.

Camera/library input is re-encoded, strips retained EXIF, limits the longest edge to 1,440 pixels and compresses toward 900 KB with a separate 2 MB hard ceiling. The saved file is a processed derivative, not the original camera asset. Metadata records dimensions, bytes, local day, capture timestamp, weight association and owner path.

Upload sequence and uncertain outcomes

The pending operation freezes a UUID file path and payload. The client checks the account/session, uploads with no overwrite, then commits metadata through the mutation RPC. A retry can tolerate the same object already existing rather than creating a different file. A database trigger serializes the per-entry count so simultaneous uploads cannot both claim the last slot.

If metadata succeeds, the client requests a one-hour signed URL. A **signed URL** is a temporary bearer capability: anyone holding it can use it until expiry, so it should not be treated as harmless public text. Private bucket and account-path policies protect normal object access. [R12]

If upload succeeds but metadata is definitively rejected, an orphan file can remain. The current mutation sender does not implement automatic rollback cleanup of every such object. There is no ongoing orphan sweeper. Whole-account deletion enumerates actual objects, which helps that separate lifecycle but does not repair an active account's storage continuously.

Deletion and viewing

Deletion commits metadata through the retry receipt first, then removes the object. If object removal fails, the pending operation can retry cleanup using the recorded path. This is staged recovery, not a transaction spanning database and files.

Weight History opens photos matching the entry ID rather than substituting other photos from the same date. Count and indicator queries are separate from image loading. The gallery renders a small nearby window; older unlinked/over-limit groups remain compatible. Weight deletion does not imply immediate erasure of independently retained photo bytes.

C18, C08, C09, ADR-003. Native orientation, permission, low-space, expired-URL and large-gallery behavior remain device checks.

35. Export: useful, but not a full backup

Profile offers a portable archive with structured JSON, spreadsheet-friendly comma-separated values (CSV), a manifest and README. Optional photo inclusion downloads processed images. Native sharing and browser download use separate adapters; ZIP creation uses the existing archive library.

exportVersion: 1 and per-dataset row counts are already present. CSV output quotes separators/newlines and prefixes formula-like strings to reduce spreadsheet formula execution. Nested values retain a structured representation in the JSON export.

What is included

The collector reads account metadata, profiles, vitals, workout sessions/sets, cardio, nutrition, hydration, saved drinks, private food profiles, recipes, photo metadata, Coach profiles/threads/messages/actions, local reminders/completions, unfinished workout/cardio/nutrition drafts, HealthKit sync state and cached vitals. It does not export passwords or protected biometric secrets as ordinary datasets.

Confirmed gaps and their consequences

The explicit dataset list omits account setup, routine preferences, Summary layout, streak rules/goal snapshots, helper drafts and durable pending-operation/outbox detail as independently recoverable records. A cached vital is not equivalent to its operation dependency chain. Therefore the archive is not a complete restore image of all intended app state.

The row loader requests 1,000 records at a time and stops on a shorter page. A smaller backend response cap can silently truncate a nonfinal table. Photo count/byte summaries have the same assumption. Parallel reads also do not form a single database snapshot: edits during collection can produce data from different moments.

Large archives are assembled in client memory, with optional file downloads adding more. A small successful export does not demonstrate that years of photos fit comfortably on an iPhone. Sharing an archive also creates a user-controlled copy outside subsequent server deletion.

Practical repair plan

Define an explicit export contract, add the missing user-facing state, use pagination that tolerates smaller pages, and include completeness warnings/count validation. Keep credentials excluded. Test with a mocked low row cap and concurrent edits. For large photos, consider streaming or a bounded asynchronous archive job only after measuring memory and timeout pressure.

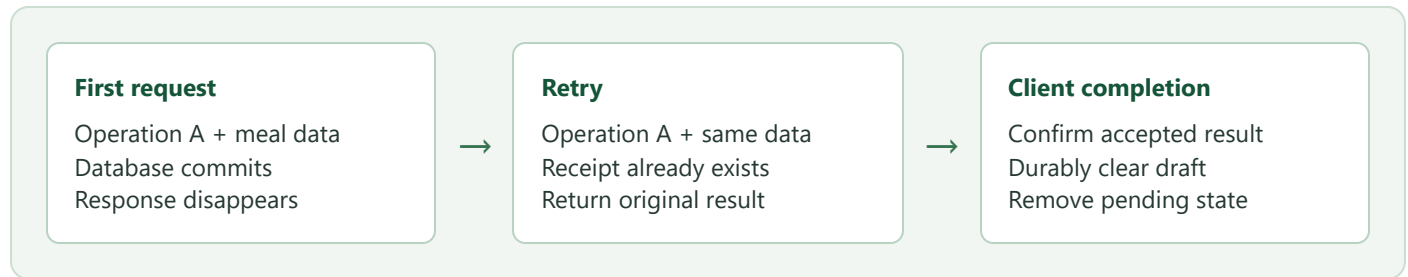
No import/restore workflow is established by the export button. Calling this a portable data export is defensible; calling it complete disaster recovery would not be.

C19. Findings are confirmed by collector and archive source; no real health export was generated during this review.

36. Walkthrough: a meal response is lost

You press Save meal. The server stores the food rows, but the phone loses connectivity before receiving the response. Retrying must distinguish that same attempt from a second meal with identical contents.

Idempotency means repeating the same identified operation has the same effect as performing it once. Sustain creates a universally unique identifier (UUID), freezes the request, and persists it locally before transmission. It also captures a canonical representation/fingerprint so changed input cannot silently replace an uncertain operation.



Inside the database transaction

`commit_health_mutation` verifies account and active session, locks the account/operation identity and checks the receipt. If the identifier exists with different content, it rejects reuse. If it is the same accepted operation, it returns the stored response.

For a new operation it locks the account's Auth user row, rechecks session state, validates allowed action/fields and expected versions, performs the writes and stores the accepted response in the same transaction. A rollback cannot leave accepted meal rows without their matching receipt. Conflict responses return before an accepted receipt is written.

The client retains accepted draft-save state until durable draft clearing finishes. Otherwise a crash after acceptance but before clearing could offer the same draft as an apparently new meal. A deliberate later identical meal receives a new operation ID and is correctly a separate event.

What this guarantee does not mean

It is not exactly-once network delivery: the request can be delivered repeatedly. It is a deduplicated effect within the mutation contract. Button disabling helps one screen avoid double taps but cannot handle process crashes or lost responses by itself. An upsert on an ID is also insufficient to preserve a frozen intent or reject a competing edit.

File uploads, OS notifications and paid AI calls do not participate in this transaction. Their retries and cleanup need separate protocols; the meal receipt cannot retroactively make them atomic.

C08, C09. Mutation model, SQL assertions and synthetic lost-response browser tests verify selected failure cases.

37. Walkthrough: two devices edit a record

Concurrency means more than one operation is in progress at overlapping times. Even a personal app has it: your phone and browser can edit the same workout, or a background sync can finish while you change a field.

Optimistic concurrency control allows editing without holding a database lock while the user thinks. The edit remembers the row version it read. The final save succeeds only if that version still matches.

Step	Phone A	Phone B
Read	Workout version 4	Workout version 4
Local edit	Change a set to 10 reps	Change a set to 12 reps
First save	Expected 4 matches; write version 5	Still has its local draft
Second save	No additional action	Expected 4 no longer matches; conflict returned
Recovery	Saved work remains	Review latest values, keep/reapply intent explicitly

The SQL function validates all versions in a grouped mutation before changing rows. Workout replacement is one transaction for parent and sets, preventing half a replacement. A deleted record's retained deletion ID prevents an old create/update from quietly resurrecting it.

Field-level comparison for goals

Goals compare changed fields with their baseline values rather than rejecting every change because some unrelated profile field moved. If A changes calories and B changes fluids, both can succeed when their own baselines remain current. If both change calories, one must review the conflict. This is **compare-and-set (CAS)** at a selected-field level.

Locks and their cost

A lock coordinates access to shared data. Sustain uses a transaction-scoped operation lock for retry identity and a per-account user-row lock for writes. Different accounts need not share the same logical account lock; one account's simultaneous writes serialize. This simplifies lock ordering and correctness, but a large import or burst can create account-specific wait time.

The default database isolation level alone does not supply all these business rules; the application protocol adds explicit checks and locks. [R04] Tests in one PGlite connection establish SQL behavior, not real simultaneous connection races. Staging tests should race independent clients, assert one winner and preserved loser state, and measure lock waits.

C08, C09; scripts/check-hardening-concurrency.mjs was inspected as a future staging tool, not run against a hosted account in this review.

38. What really works offline

Offline-first means the application can accept meaningful work locally and reconcile it later under an explicit conflict policy. Merely keeping text in a form is useful, but it is a weaker promise. Sustain intentionally provides different contracts for different features.

Feature/state	Without connectivity in an existing process	What still needs connectivity
Manual weight/BP	Durable local cache/outbox save, edit and deletion intent	Cloud acknowledgment and remote reconciliation
Workout/cardio/meal	Account-local unfinished drafts; durable uncertain operation state where attempted	Confirming completed server history
Fluid/goal/history mutation	Retained pending operation and reviewable input	Authoritative acceptance/conflict result
Summary/vital display	Available local readings/state may be shown	Missing remote domains, fresh complete history
Reminder definitions	Local settings and OS scheduling subject to permission/capacity	No cloud service required for ordinary local delivery
HealthKit import	Existing cached data is available	Implemented import requires successful pre/post cloud sync
AI/barcode	Existing manual inputs/definitions remain useful	New external provider result
Fresh authentication	Remembered identity can be displayed	Password sign-in or fresh Face ID session exchange

Example: an unfinished workout on a train

You can preserve exercises and sets in the local draft while the train has no signal. Pressing Save does not turn that draft into confirmed cloud history until the server responds or a retry receipt confirms acceptance. Conversely, a manual weight can be accepted into the durable local vital store and visibly remain pending sync.

The outbox preserves operation dependencies and original versions. A cache refresh must merge around unsent local changes, not overwrite them. Signing into another account does not send those changes under the new identity; pending work belongs to the original account.

Why not extend the outbox everywhere immediately?

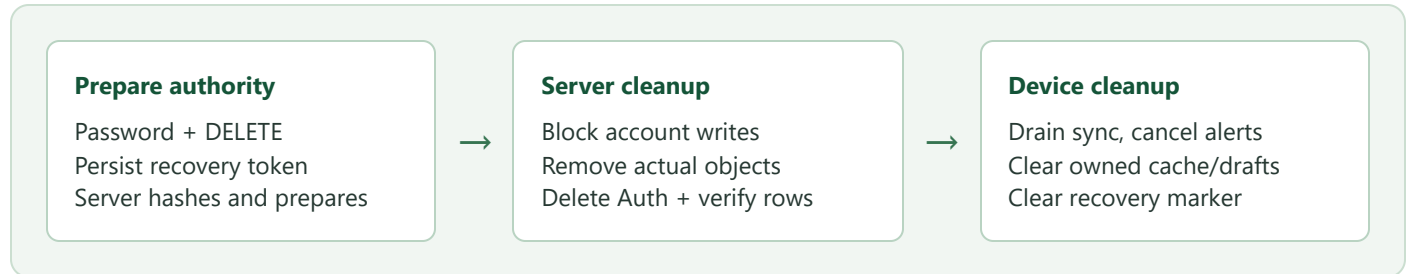
Meals involve labels/snapshots, workouts replace parent/child graphs, photos span object storage, and goals have effective dates. Each needs edit/delete/conflict/recovery semantics, not just a generic "retry later" loop. The current scope reduces solo-maintenance burden while prioritizing quick vital entry.

A broader offline roadmap should start with a specific user need, define local success and conflict behavior, and test process death at every durable boundary. Cold-start offline access would also require reconsidering the process-only session policy, not merely adding more cached tables.

C04, C08, C12-C19. Preserving drafts, confirming records and retaining an authenticated process are intentionally separate claims.

39. Walkthrough: deleting an account

Account deletion crosses authentication, database rows, files, notifications and local storage. A single database transaction cannot erase all of them. Sustain uses a staged recoverable workflow, sometimes called a **saga**: several operations have explicit progress and retry rules.



The client creates a recovery capability from two UUIDs and stores a pending marker. The endpoint independently verifies bearer identity and password before preparation. A service-only SQL function serializes against writes, records the hashed token, revokes device credentials and removes sessions. Covered policies and guards reject further account activity.

Ordinary authentication can no longer be relied on after preparation. The saved capability authorizes status/continuation for that prepared job. The endpoint's documented gateway-JWT exception supports this session-free recovery; it is not permission for arbitrary unauthenticated deletion. Initial preparation proves identity/password, continuation proves possession of the secret token.

Cleanup enumerates actual Storage objects using ownership/path information, including files missing photo metadata. It removes bounded batches, deletes the Auth user and relies on owned-row cascades. The finish function checks that the relevant account/data/object remnants are gone before recording completion and removing account identity from the retained receipt.

Device cleanup waits for owned sync activity, cancels owned routine/custom requests and delivered notifications, removes local readings/drafts/account keys/credentials/private cache, signs out and clears the marker only after the appropriate completion boundary. Failure leaves a resumable state rather than pretending success.

An unstarted token can be sealed as cancelled so a delayed prepare request cannot start later. Once preparation has begun, leaving/signing out is possible but cannot reverse data already removed. Missing-function/service errors must remain distinguishable from an unstarted request.

This does not remotely erase an offline second phone, user-exported files, external provider records or all backup copies. Those require separate retention/disclosure policies.

C20, C09, C17. Historical hosted disposable-account deletion is documented; this review ran only isolated/synthetic checks.

40. Decision: React Native and Expo

Requirement: an iPhone-first product with HealthKit, camera, Face ID and notifications, manageable from the existing Windows development setup, while gaining React/TypeScript experience.

Selected approach: React Native components, shared TypeScript models, Expo Router, native Expo modules and a custom iOS development client. Browser adapters provide a fast preview and synthetic end-to-end test surface. EAS supplies a practical remote native-build path.

Alternative	Why it is credible	Tradeoff in this project
SwiftUI	Apple's declarative native UI framework, direct platform APIs and tooling	Strong choice for permanent iPhone-only scope. Would emphasize Swift/platform experience and require a different development workflow; less shared React/browser code.
Flutter	Cross-platform UI framework with Dart and its own widget/rendering ecosystem	Viable shared-mobile approach. Introduces a different language/ecosystem from the stated learning goal; native integrations still need evaluation.
Web app / progressive web app	Browser-delivered application, simpler distribution and web skill transfer	Useful companion possibility, but does not provide the same native HealthKit path and platform interaction guarantees.
Bare React Native	Greater direct native project control	More native configuration/build maintenance; Expo already supplies much of what this app needs.

Why the choice is reasonable

Reusable forms, domain arithmetic, validation, repository contracts and tests span features. Native adapters expose capabilities a browser-only product cannot simply assume. A single feature-oriented TypeScript codebase reduces context switching for one maintainer. These are technical reasons independent of job-market preference.

Accepted costs

Dependencies and native bridge compatibility must move together. Browser CSS/DOM behavior differs from native text hosts, keyboard insets and accessibility. The native Text rendering issue covered by regression tests is a concrete example of why web success is insufficient. Expo modules and SDK upgrades still require native builds and device checks.

Reconsider when: most work becomes deeply platform-specific, native performance/interaction limits persist after profiling, or the intended product changes to browser-first. There is no measured benchmark here proving React Native faster or cheaper than SwiftUI/Flutter; the selection is about requirements, workflow and maintainability.

C01-C05; user-stated learning motivation plus retrospective engineering analysis. No undocumented framework bake-off is claimed.

41. Decision: managed backend and SQL

Requirement: shared durable history, account isolation, relational records, protected provider credentials and recoverable mutations without operating a large backend stack alone.

Documented choice: ADR-001 selected hosted Supabase and reviewed SQL migrations without requiring Docker/WSL2 on the development machine. Current code uses direct authenticated reads, guarded transactional SQL writes and a small set of Edge Functions for privileged/external workflows.

Alternative	What it changes	Benefits and costs
Local SQLite only	No account/cloud backend	Excellent personal/offline simplicity; multi-device recovery, account lifecycle and protected external calls must be added separately.
Custom API + PostgreSQL	All clients call an owned application service	Explicit contracts and policy layer; another deployment, authentication integration, observability and scaling responsibility.
Firebase/document store	Documents and provider-specific access/sync model	Viable managed approach; relational workout sets, ownership links and historical reporting require different modeling/rules.
Self-hosted services	Operate the database/Auth/storage infrastructure	More control and potential portability; patching, backups, availability and incident response shift earlier onto the developer.

Why relational data fits

Sessions own sets, nutrition entries reference private definitions while retaining snapshots, and operations own receipts. Foreign keys, uniqueness, transactions and SQL reporting directly express these relationships. JSON fields cover flexible nested snapshots where a rigid column for every optional input would be cumbersome.

Saying "SQL scales better" would be an unjustified universal claim. The useful claim is that PostgreSQL matches this data's relationships and consistency needs, while managed services reduce infrastructure work.

Why functions live in the database

An RPC is a remote procedure call: the client asks a named server function to perform a defined operation. The SQL RPC keeps data changes and receipts inside one transaction. It also centralizes checks that cannot safely live only in a phone UI.

The downside is substantial business logic in migrations and platform-specific session/Auth contracts. A dedicated API becomes more attractive with multiple independent clients, a larger team, complex integrations or difficult contract versioning. Adding it would not remove the need for database constraints. Repositories localize migration work, but do not make vendor replacement free.

C08, C09, C21, ADR-001. Managed-platform responsibilities: R01-R03. Alternatives are retrospective analysis unless explicitly documented.

42. Decision: consistency and historical data

The requirement is not merely storing a row. A user must be able to distinguish a repeated network attempt, a competing edit, a corrected definition and a newly recorded event.

Choice	Benefit	Accepted cost / credible alternative
Operation IDs + receipts	Lost-response retries do not duplicate accepted mutations	Persistent operational records and recovery UI. Button disabling or plain upsert is simpler but covers fewer failure modes.
Version/CAS checks	Competing edits are visible	User must resolve conflicts. Last-write-wins is simpler but can silently discard changes.
Historical snapshots	Label edits do not rewrite meals; dated goals preserve meaning	Duplicated data and explicit correction workflows. Live joins save space but change past interpretation.
Derived streaks	Backdated edits/deletes can recalculate accurately	Historical reads and client computation. Stored counters need invalidation and reconciliation rules.
Vital outbox only	Prioritizes useful durable offline entry	Uneven offline capability. Full offline-first architecture adds domain-specific merge and lifecycle work.

Why these matter for one user

A personal phone can lose a response or crash. The same person can edit from two devices. A reusable label can be corrected. These are correctness problems independent of a large user base. Reliability investment is justified by data semantics, not imaginary millions of requests.

Some complexity deserves reevaluation. Account-sized JSON inside SQLite is simple to migrate and transact, but it rewrites and parses more data as history grows. A normalized local database would support indexed subsets and smaller changes, at the cost of local schema migrations and more query code.

Receipt/tombstone accumulation preserves retries and deletions for long-offline clients. Deleting old operational records blindly could permit duplicates or resurrection. A retention design needs a supported retry window, cursor expiry/reset and a full reconciliation protocol before cleanup can be safe.

Reconsideration triggers

Measure Summary read bytes, client memory, local write duration, historical recomputation time and per-account lock waits. If they exceed a chosen usability budget, add pagination, incremental aggregates or normalized local rows first. A server-maintained summary must invalidate for every relevant source/rule/edit/delete, not simply increment a counter on save.

C08-C11, C14-C15. These are engineering tradeoffs grounded in implemented protocols; performance benefits remain hypotheses until measured.

43. Decision: preferences, reminders, and state

Local versus cloud reminder architecture

Local schedules avoid a server job runner, push-token lifecycle and remote delivery dependency. Stable OS repeats work without the JavaScript process. This fits personal use and a manageable operating budget. Costs include device-only settings, limited pending-request capacity and finite replenished windows for some schedules.

An alternative would synchronize reminder definitions and send pushes from a server. That would support cross-device management and centrally evaluated conditions, but requires user/device/timezone ownership, permission/token changes, deduplication, retries and observability. It would still not guarantee a person saw a prompt or took medication. Adopt it for an explicit product requirement, not simply because more accounts exist.

Device-local preferences versus synchronized settings

Summary order and unfinished drafts are cheap and responsive locally. A second phone having a different layout can be acceptable. If cross-device continuity becomes important, define which preferences are account-wide and which are per-device, version them, and resolve conflicts. Do not synchronize native notification request IDs or HealthKit anchors as if they were portable preferences.

Existing React state versus another library

Hooks/context and feature repositories keep dependencies modest and preserve explicit save/draft boundaries. A query library could standardize cached fetches, deduplication and invalidation as screens grow. A global state library could make complex shared state easier to inspect. Neither supplies idempotent database writes or correct offline merging automatically.

Reconsider when duplicated fetching/invalidation causes demonstrable bugs or state flows become difficult to understand. Introduce one responsibility at a time; do not replace working domain models merely to standardize a fashionable stack.

Process-only authentication versus convenience

The chosen session policy reduces persisted ordinary tokens but requires online fresh entry. Custom Face ID credentials restore convenience with considerable protocol complexity. A provider-supported persisted session protected by native access would be simpler; password-only would minimize custom auth but increase friction. This is a product/security/maintenance tradeoff, not a scalability optimization.

For public use, prioritize a security review and explicit offline-entry expectations before extending custom authentication. Choose the simplest supported lifecycle that meets those requirements; preserve account isolation and failure visibility whichever mechanism is selected.

C04-C08, C16-C17; ADR-009/010. Alternatives are retrospective analysis, not a claim that each was prototyped.

44. Verification and what it establishes

This review ties results to application source 088f60c. Checks use synthetic data and isolated local services. No hosted account, paid model, real health record or physical iPhone was exercised.

Category	Evidence and limits
Lint / TypeScript	Root npm run check passes. Static checks catch selected code/type issues, not runtime behavior.
Unit/component-style	240 app tests and 66 Edge tests pass. Pure rules, storage state and mocked/native-host boundaries; not actual device APIs.
Isolated SQL	All 36 migrations apply in PGLite; hardening, onboarding, streak and Summary/deletion assertions pass. Auth/Storage schemas are minimal stand-ins; no real concurrent connections.
Browser, account/lifecycle	40 selected auth, entry, onboarding, Profile, reminder and deletion cases pass. Synthetic API fixtures, not hosted RLS or native delivery.
Browser, other features	39 of 45 initial selected cases pass. Six failures are described below; do not interpret the run as fully green.
Native export/config	Historical source-snapshot records report iOS/Hermes export and config inspection. Not rerun here; neither proves installed-device behavior.
Hosted/load/device	Historical hosted checks exist. Fresh deployment state, genuine concurrency, workload capacity and physical-device acceptance remain unverified.

Current test-maintenance findings

One planner test expects the obsolete accessibility label Placeholder on the designed Soon tab. Five date/helper tests freeze the wall clock before login and fail to reach accessible app controls behind the entry transition; unchanged reruns reproduce them. All eight date/helper cases pass in a separate Reduce Motion diagnostic, isolating the interaction with animated entry. The original runs remain failed.

The initial sandboxed test command could not launch subprocesses (EPERM); rerunning with permitted subprocess execution produced the passing unit/static results. This is an environment failure, separate from the browser assertions above.

What to prove next

Use two disposable staging accounts for malicious cross-owner reads/writes/storage paths. Race independent connections for versions, quotas and last photo slot. Kill/restart the device around queue commit, receipt acknowledgment, credential rotation and deletion stages. Validate real notification delivery, keyboard, Face ID, HealthKit, VoiceOver and Dynamic Type. Load tests need explicit data volume, concurrency and error budgets before any capacity claim.

C22; exact log paths and diagnostic outcomes are on page 62. Tests establish their exercised scenarios, not complete correctness or production readiness.

45. Findings: correctness and launch gates

Priorities below are review recommendations, not silently applied changes. P1 means resolve before making the relevant public promise; P2 means near-term hardening. A verification gap is not automatically a proven runtime defect.

P1 - Export completeness and pagination

Confirmed limitation/conditional defect. The collector omits several user-facing state categories and stops early under a lower server row cap. Users could believe an incomplete archive is complete. Define the export contract, add intended datasets, use robust pagination and completeness checks. Cost: schema/version work and larger archives. Verify with more than one page, a deliberately low cap, missing categories and concurrent edits. Evidence: C19.

P1 - Native and live isolation acceptance

Verification gap. Browser fixtures cannot prove installed Face ID, HealthKit, OS delivery, keyboard/accessibility or deployed RLS/Storage policies. Before beta claims, run a physical-device matrix and two-account disposable staging checks, including recovery failures. Cost: device time, isolated environment and repeatable scripts. Success is recorded reproducible evidence, not another web screenshot. Evidence: C22.

P1 - Public release configuration and disclosures

Confirmed configuration mismatch plus unknown hosted state. HealthKit purpose text mentions workout reads absent from the reader; native display name remains repository branding; tablet support is enabled without a demonstrated tablet release scope. Align intended product scope, purpose strings, icons/name, privacy/support links, SMTP and callback settings. Cost: native build, metadata and hosted setup review. Verify a clean installation and real email/reset flow. Evidence: C01/C13; R05-R10.

P1 - Authentication and recovery capability review

Accepted complexity with a verification gap. Custom credential rotation deliberately fails closed after lost responses, and generic protected-storage fallback weakens assumptions about recovery markers. Review platform storage behavior, threat model and supported standard alternatives. Cost: focused security expertise and device failure tests. Verify no secret enters ordinary storage unexpectedly and every failure has a usable recovery route. Evidence: C06/C20.

P2 - Regression harness reliability

Confirmed test failure. Obsolete selectors and fixed-time entry behavior prevent six original feature cases from passing. Keep calendar tests deterministic while allowing animation time to progress, and update selectors to current semantics. Cost: small test maintenance; no product behavior change is implied. Verify original affected suites and a separate real-clock motion case. Evidence: C22.

46. Findings: reliability and growth work

P2 - File reconciliation and lifecycle

Confirmed recovery gap. A file uploaded before rejected metadata can become orphaned; no automatic sweep is present. Add conservative reconciliation with an age threshold and pending-operation protection, plus storage metrics. Cost: privileged job, lifecycle tests and deletion safeguards. Verify simulated rejected uploads, lost responses and valid in-flight objects are distinguished. C18/C08.

P2 - Reminder promises and cross-device expectations

Product limitations. Some schedules end after 14 occurrences or 28 days without reopening; custom schedules can survive logout; preferences/completions are device-local. Clarify these semantics and test capacity/DST/travel. Only add cloud scheduling if indefinite centrally managed behavior is required. Cost: modest UX/device testing first; server push is a larger architecture change. C16/C17.

P2 - Observability, recovery, and AI evaluation

Verification gaps. No measured public error budget, restore drill or representative paid-model quality evaluation is established. Add redacted operational metrics, budget alerts, backup/object recovery drills and a consented evaluation set. Cost: operations tooling and controlled evaluation spend outside this review. Verify restored references and deletion guards, quota behavior, model validity and user correction rates. C21/C22; R01/R02/R13.

P2 - Long-history and local-memory profiling

Speculative growth risk grounded in code. Summary loads historical arrays, exports assemble archives in memory and SQLite rewrites account payloads. Test years of synthetic history on lower-memory devices. Optimize bounded queries, aggregation or normalized local rows when measured budgets fail. Cost: query/migration/invalidation work. Verify p95 latency, peak memory, response bytes and cold-start stability. C08/C10/C19.

P3 - Operational record retention

Accepted tradeoff. Receipts and deletion markers accumulate until account deletion. Introduce garbage collection only with retry/cursor expiry and reset/full-reconciliation semantics. Cost: a protocol change supporting older clients. Verify a long-offline installation cannot resurrect deleted data or duplicate a save after cleanup. C09.

P3 - Compatibility code and module size

Maintenance risk. Broader Coach, retired streaks and legacy reminder/food-day concepts remain. Document active boundaries, remove unused UI only with compatibility/data migration plans, and extract cohesive modules when edits become hard to test. Cost: moderate refactoring with limited user-visible benefit. Verification must preserve supported old-client contracts, not merely reduce file count.

Prioritize trustworthy behavior and evidence first. No Redis, microservices, queue or backend rewrite is a launch prerequisite inferred from this review.

47. Deploying the complete product

Deployment means making a particular software/configuration version available in its intended environment. Sustain has two major delivery tracks: installing the mobile application and operating its remote services. Publishing one does not deploy the other.

Component	Where it runs	Release responsibility
iPhone UI/domain/native adapters	Installed signed application	Build, test and distribute a native binary.
Authentication	Supabase Auth	Configure providers, email, session policy and allowed links.
Records and SQL functions	Hosted PostgreSQL	Apply reviewed migrations, grants, constraints and policies.
Five server functions	Supabase Edge runtime	Deploy reviewed code with correct authentication settings and secrets.
Progress-photo bytes	Private Supabase Storage bucket	Configure bucket bounds/policies and file lifecycle/recovery.
AI / barcode providers	External services	Credentials, consent, quota, compatibility, attribution and spending controls.
Local notifications / HealthKit	User's iPhone	Native capabilities and permissions; schedules/import logic ship in app.
Browser preview	Development/test browser	Useful engineering surface; no production web companion release is established.

Environments should mean separate data boundaries

Development supports rapid changes, staging rehearses releases using synthetic/disposable accounts, and production holds real user records. Separate backend projects, credentials, email configuration and storage prevent a test migration or fixture from affecting personal data. Do not copy real health data into staging merely for realistic volume.

The repository has EAS development, preview and production build profiles. Development uses a separate .dev bundle identifier and healthapp-dev scheme; production uses com.jalen.healthapp and healthapp. Preview shares the nondevelopment identity unless deliberately configured otherwise. Profile names alone do not prove independent backends or installable side-by-side staging binaries.

Public Supabase URL/key values are build configuration, not administrative secrets. Server-role and provider keys belong in server secret storage. Inventory each environment explicitly, including callback schemes, bucket policy, Edge gateway settings and model enablement. The linked development project and historical deployment records do not prove a separately hardened production environment exists.

C01, C21-C22; R01/R02 and R05/R06. This is an actionable deployment plan only; no release commands were executed.

48. Personal development to invited beta

Phase 1: personal development

Keep the current feature modules, hosted architecture and local native development client. Use reviewed migrations and synthetic tests. Native HealthKit and Face ID require the rebuilt custom client, not Expo Go. Preserve personal data while using a separate staging project for destructive failure tests.

Must complete: install the current native modules, verify physical permissions/entry/keyboard/reminder behavior, and record which client build matches which backend migration/function versions. Verify auth email links and save/recovery with disposable accounts. **Optional:** a public web host or cloud notification service. Neither is necessary for personal use.

Phase 2: a small invited beta

Choose a bounded group, for example 20-50 consenting testers as a planning assumption. Rehearse migrations and function deployment against staging, then provision production configuration. Use a dedicated email delivery provider through Simple Mail Transfer Protocol (SMTP); Supabase's default email service is restricted and is not intended for production. Test verification/reset from non-team addresses. [R07]

Build a signed iOS binary through the production-appropriate EAS profile, with Apple distribution credentials, matching bundle identifier and capabilities. Upload through EAS Submit/App Store Connect; upload is not the same as App Store approval. TestFlight provides controlled distribution and feedback. Apple documents up to 90 days per beta build and review for initial external distribution. [R05, R06, R08]

Beta exit evidence

Before inviting users, require passing staging ownership/retry/deletion checks, the export contract either repaired or accurately bounded, privacy/support information, a tested backup/object recovery procedure, operational alerts and a way to disable costly AI generation. Verify the exact production binary with its production callback scheme and backend.

During beta, collect redacted crash/error rates, save failures, pending-age distribution, sync conflicts, notification complaints, memory/performance on long synthetic histories and AI correction feedback. Do not collect raw meal photos or health values just to debug a metric.

Can remain: managed Supabase, SQL mutations, local reminders and feature-local state. **Optional:** paid performance tooling or extra automation. **Proceed only when:** critical recovery flows are reproducible, support ownership is clear, and actual beta workload fits explicit reliability/cost budgets.

C01/C22; R05-R08. Proposed phases are recommendations, not evidence that TestFlight or production has already been configured.

49. Public release and platform requirements

Requirements verified from official documentation

Apple's current published minimum requires App Store Connect uploads to be built with Xcode 26 or later using the iOS 26 SDK or later, effective April 28, 2026. Select an EAS build image meeting current submission rules and recheck the requirement at release time. JavaScript export does not establish the native SDK used in a signed binary. [R09]

Account-creating apps must provide in-app account deletion. Sustain has the workflow, but release acceptance should verify it end to end on the deployed backend and device. A functioning button is not proof of complete cleanup. [R17]

Apple requires accurate privacy information, including relevant third-party practices. Its review guidance requires an accessible privacy policy, accurate metadata and review access, and limits health-data use for advertising/marketing/data mining. Review HealthKit-specific handling and backup configuration against the actual data flow. [R18, R19]

Sustain-specific release work

Align display name, icons, splash, permission explanations and store screenshots with Sustain. The source config still uses HealthApp names and enables tablet support; either validate that intended scope or narrow it deliberately. The HealthKit purpose text must describe weight/BP/pulse reads accurately. Camera/library explanations should cover actual barcode, meal-image and progress-photo use.

Exercise consent and denial paths for HealthKit, camera/photos, Face ID and notifications. Review third-party SDK privacy manifests/required-reason APIs in the built app. Verify signup/confirmation/reset links on a fresh installation, same-account recovery, no-session reopening and wrong-account reminder taps.

Publish support contact, data handling/retention, deletion limitations, AI processing disclosure and a clear product scope. Do not infer a medical regulatory classification or compliance status from the word wellness. Broader claims, markets and intended uses require their own qualified review.

Public-launch gate

Require the beta evidence plus operational monitoring, recovery rehearsal, abuse/cost controls, accurate export semantics, old-client compatibility checks and a release rollback procedure. Keep backend services available for store review and supply a suitable review account/demo path with synthetic content. Optional broader features can remain deferred; a Coming Soon tab is not evidence of a completed feature for marketing.

R09, R17-R19 verified 23 September 2026. Requirements above are distinguished from the report's recommended engineering gates; no compliance certification is asserted.

50. Releases, compatibility, and rollback

Sustain's `app.config.ts` sets `updates.enabled: false`. The current production release mechanism is a new native build, including JavaScript changes delivered to installed release clients. Reloading Metro helps a development client; it does not update every installed user's app. Enabling over-the-air updates would be a deliberate future configuration/security/compatibility decision. [R20]

Change	Required delivery path
UI/domain JavaScript in current release setup	New tested native build and distribution.
Native module, permission text, entitlement, splash	New native build; verify actual generated native configuration.
SQL table/constraint/function/policy	Reviewed database migration applied to intended environment.
Edge handler/provider integration	Server-function deployment; verify gateway/auth settings.
Provider key/model/enablement/email setting	Controlled server/environment configuration, with validation.
Privacy/store metadata	Store/support updates; some changes also require app changes.

Expand before removing

An older installed app continues sending its old contract even after the backend updates. Use an **expand-and-contract** release: first add compatible columns/functions/defaults; then deploy handlers supporting both supported contracts; test older and newer clients; distribute the new binary; only remove old paths after a deliberate support window and migration plan.

This pattern matters for the retained Coach request normalization, retired streak rows, reminder schema migration and additive fluid policy defaults. Deleting compatibility code because the newest UI no longer uses it can break an installed client or historical record.

Rollback is not simply undoing SQL

A function version can often be rolled back if its data contract remains compatible. Dropping a column or undoing a data transformation may lose data or make newer writes unreadable. Favor forward corrective migrations and compatible feature disablement over destructive schema reversal.

Keep build IDs, source commit, applied migrations, function versions and secret/config changes in a release ledger. Retain a tested prior binary and server artifacts. Mobile review/distribution takes time, so the backend should tolerate an overlap period. A database restore can invalidate local sync cursors; the current app fails visibly on inconsistent cursors rather than automatically rebuilding them. Recovery must include client reconciliation.

C01, C08/C09, C21; R20. No over-the-air release capability or automatic whole-system rollback is claimed.

51. Operating the service responsibly

Observability means knowing what the system is doing through metrics, logs and traces. Start with actionable redacted signals: operation success/conflict/error counts, latency percentiles, pending save age, sync backlog, function failures, photo cleanup failures, email delivery failures and provider quota/cost usage.

A percentile describes a distribution: p95 save latency is the time within which 95% of measured saves completed. It is more useful than one average that hides a slow tail. Choose a target before testing; do not retrospectively call any measured number acceptable.

Recovery objectives

Define a **recovery point objective (RPO)**, the maximum acceptable lost data interval, and a **recovery time objective (RTO)**, how long restoration may take. For a small beta, proposed targets might be RPO 24 hours and RTO one business day, subject to user expectations and actual plan capabilities. These are proposed budgets, not achieved guarantees.

Supabase database backups do not include Storage object bytes. A restored photo metadata row can therefore point to a missing JPEG. Maintain a compatible object backup/retention plan and test restoration into an isolated environment, including Auth/session behavior, object policies, deletion markers and local cursor recovery. [R13]

Release and incident checklist

Before release, review migrations/grants, run synthetic checks, confirm staging isolation, test supported old/new clients and verify production secrets without exposing values. After release, watch error and spending signals, confirm auth-email delivery, and check the first real support reports against redacted operation IDs.

When an incident occurs, contain it with the smallest reversible action: disable AI generation, stop a bad function rollout or pause a failing feature. Preserve diagnostic metadata and avoid asking users to send raw exports or credentials. Communicate affected behavior and recovery steps; account deletion and sign-in failures deserve a clear support path.

The current code has useful bounded error handling but no demonstrated public monitoring/restore program. Hosted providers handle infrastructure portions; the developer still owns application configuration, query efficiency, sufficient resources, retention decisions and incident response. [R02]

A small runbook and tested recovery exercise are higher-value first operations investments than adding another service whose failures are also unobserved.

C22; R02/R13. Operational targets and procedures are recommendations. No backup restore, alert drill or uptime measurement was performed here.

52. Scaling starts with workload assumptions

Scaling means handling more work while retaining acceptable reliability, responsiveness, cost and operating effort. Three dimensions differ: **more accounts**, **more history per account**, and **more activity at the same time**. An account can be slow with years of history even when only one person is online.

The following are deliberately hypothetical 30-day planning scenarios, not supported-capacity claims. Each active user averages 12 logical record actions/day, 0.1 photo/day at 0.75 MB, and 0.2 AI requests/day. A logical action can create several rows and HTTP requests; workouts and BP are examples.

Scenario	Beta	Early public	Larger measured demand
Daily active users	30	500	5,000
Logical actions/day	360	6,000	60,000
Actions/30 days	10,800	180,000	1.8 million
New photos/30 days	90	1,500	15,000
New photo storage	67.5 MB	1.125 GB	11.25 GB
AI requests/day	6	100	1,000
Illustrative peak active sessions	5	50	500

Convert averages into tests

If the 500-user case puts 10% of daily actions into a ten-minute burst, that is 600 logical actions/600 seconds, or one action/second on average during that burst. The 5,000-user case would be ten actions/second under the same assumption. Request fan-out, retries, imports and correlated AI calls can multiply actual service work. These figures are test inputs, not throughput predictions.

Photos accumulate unless deleted; downloads can cost more bandwidth than uploads. An older user's Summary may scan many years even if today's write count is small. Simultaneous sessions also need not be simultaneously writing, so do not convert session count directly into database connections.

Test ordinary usage, a burst, one account importing thousands of readings, multiple devices editing one account, and users with long histories separately. Measure response distributions, resource pressure, correctness and spend. A design with account isolation has a foundation for multiple users; only this evidence can support a capacity statement.

All numbers on this page are planning assumptions; arithmetic is reproducible and no load test was executed.

53. Database, history, and client pressure

Likely first investigation: data depth

Summary's historical streak evaluation can read full relevant histories and aggregate on the client. Keyset pagination avoids truncation but still accumulates arrays. Export downloads many datasets and optional images. Native vital storage serializes account-sized cache/outbox JSON. These are concrete paths to test before assuming the database needs a rewrite.

Start with one, five and ten years of synthetic readings, meals and sets. Measure total fetched bytes, query count, p95 screen readiness, parsing time, peak memory and local save time. Inspect database query plans for owner/time filters, join/selectivity and sort work. Use query statistics and slow-query analysis; add indexes only for observed access patterns.

Incremental improvements

Project only needed columns, query visible time windows, page long lists and defer noncritical data. Move heavy repeated historical aggregation to a bounded SQL query if profiling justifies it. Cache derived results only with explicit invalidation for edits, deletions and dated-rule changes. Normalize SQLite cache/outbox rows if full-payload rewrites dominate local work.

These changes preserve the existing account model and managed architecture. They introduce query/schema/invalidation work but less operational burden than a new service.

Concurrency and connections

Measure per-account lock waits during imports and simultaneous edits. The user-row lock serializes one account's mutations; adding accounts does not inherently create one global application lock. Database CPU, input/output, pool pressure and shared indexes can still become global bottlenecks.

The mobile app uses Supabase HTTP services, not a permanent direct SQL connection per phone. If future workers or custom APIs open SQL connections, choose pooling/transaction behavior deliberately and test connection limits. More compute can be a practical incremental step once query inefficiency and workload shape are understood.

Triggers for architectural change

A dedicated API is justified by complex contracts/team boundaries or integrations, not just user count. A job queue is justified when work reliably exceeds interactive budgets or needs durable background completion. Read replicas fit measured read pressure and consistency tolerances; they do not fix client memory or per-account write conflicts. Partitioning/sharding should follow measured table/index/maintenance pressure, not be an initial beta requirement.

C08-C11/C19; R01-R04. These are testable hypotheses and incremental options, not measured bottlenecks.

54. External limits and workload-based cost

Database efficiency is only one cost dimension. AI calls, photo storage/egress and email can become operational limits before row counts do. Avoid quoting a monthly total without a measured workload and current provider plan.

Cost formulas

Structured storage: accumulated rows $\times$ average row bytes, plus indexes, receipts and deletion markers. A logical workout produces a session and many sets; use measured row size and retention rather than counting button presses.

Photos: retained photo count $\times$ mean processed bytes for storage; total viewed/exported downloads $\times$ bytes for egress. Compression reduces cost but does not cap lifetime storage. Three photos per weight entry can still produce a large archive.

AI: requests by feature/tier $\times$ (input tokens $\times$ input rate + output tokens $\times$ output rate + applicable image/tool costs). Add failed billed calls, not only successful reviewed results. Track tokens and model configuration; app quota refunds do not refund provider usage.

Other services: database compute/backup/egress, function invocations, Auth monthly active users, SMTP messages, build/distribution tooling and observability. Use official current rates when budgeting; this report intentionally gives formulas rather than stale dollar estimates.

Practical limits to monitor

Supabase documents Edge memory, CPU, wall-clock and idle-time limits; at review time these include 256 MB memory, 2 seconds CPU/request, 150 seconds idle timeout and plan-dependent worker duration. Sustain's bounded provider timeouts do not make every multi-step function immune to those limits. [R21]

Open Food Facts currently documents 15 product reads/minute/IP and separate search limits. Centralized proxy traffic can share that ceiling; cache misses and bursts need monitoring even with few active users. Honor attribution/licensing and avoid assuming all catalog traffic is unlimited. [R14]

Per-user AI quotas bound one account, not the total project bill or abuse from many accounts. Use server enable flags, global usage/spend alerts, provider-side hard controls where available, rate limiting and a reviewable activation plan. Confirm whether each provider's budget control alerts or actually stops requests.

Notifications currently consume device scheduling capacity rather than a server delivery bill. Switching to remote push adds token storage, scheduling, retries and support costs even if the push transport itself is inexpensive.

C17/C18/C21. No price, cost saving, model quality or maximum throughput is asserted without measurement.

55. Staged growth recommendation

Small invited beta

Use the 30-active-user scenario as a test plan, not a sales claim. Keep one managed backend, SQL functions, local notifications and the native app. Measure crashes, save/error/conflict rates, pending age, auth-email delivery, history load and AI spend. Run device recovery and staging isolation/concurrency tests before inviting testers.

The first costs should be appropriate managed resources, email delivery, recovery capability and modest monitoring. Proceed only after critical flows are reliable and support can diagnose failures without collecting raw health data. A separate web companion and remote reminder server can wait.

Early public usage

Use observed beta activity to replace the 500-user assumptions. Load-test likely bursts and deep histories. Add bounded reads, robust exports, photo reconciliation, operational alerts and abuse controls. Raise database resources when measured CPU/I/O/connection pressure persists after sensible query work.

Set proposed service budgets, for example p95 confirmed save below two seconds under the tested network/workload and fewer than 0.5% unexpected server errors. These are starting product targets, not achieved results or universal standards. Separate provider-generation latency and offline pending time from ordinary save latency.

Larger usage with measured bottlenecks

Use the 5,000-user scenario only as an illustrative next test. If interactive exports exceed memory/time budgets, introduce a durable asynchronous export job with status and expiry. If repeated aggregates dominate reads, add maintained summaries with reconciliation. If centrally managed reminders become a product requirement, build a remote notification system with explicit device/timezone semantics.

Each architectural shift creates a new consistency and operations burden. A queue needs retry/deduplication/dead-letter policies; a cache needs invalidation; additional services need authentication, deployment and monitoring. Redis, Kubernetes, microservices and sharding are not prerequisites inferred from the current app.

Recommended first public architecture: retain React Native/Expo plus hosted Supabase, use a separate staging project, distribute through TestFlight before the App Store, keep AI optional and controlled, and operate local reminders with accurate limits. Prioritize export correctness, native/security verification, recovery and observability before scaling infrastructure.

Launch blockers: unsupported privacy/permission claims, unverified critical account isolation/deletion, missing real email/recovery paths and unreliable essential flows. **First optimizations:** bound heavy history reads, profile local payloads/exports, reconcile photos and tune indexes from measured plans. More infrastructure follows evidence.

Recommendations synthesize the reviewed implementation and official platform guidance. Multiple accounts are feasible in the data model; capacity is not yet demonstrated.

56. Interview answers: project and architecture

Why React Native and Expo?

Short answer: "I built Sustain for my own iPhone workflow and wanted transferable React/TypeScript experience. React Native let me reuse component and domain patterns while still reaching HealthKit, Face ID and the camera. Expo reduced native setup/build work, which mattered for a solo project on Windows."

Deeper: screens, validation and calculations are shared; native adapters and a custom build supply device capabilities. Browser tests accelerate feedback but do not replace native testing. **Follow-up:** why not SwiftUI? It is a strong iOS-only alternative; this choice balanced workflow and learning with technical fit. No performance superiority was measured.

Why Supabase and PostgreSQL?

Short answer: "My records have relationships and transactional rules: workouts own sets, meals preserve snapshots, and retries need receipts. PostgreSQL expresses those directly. Supabase added managed authentication and private storage without requiring me to operate a separate server for every read."

Deeper: RLS protects ordinary account access; guarded SQL functions enforce mutations and receipt atomicity. Edge Functions handle secrets and external services. **Follow-up:** why not Firebase? A document model is viable, but would express these joins/ownership rules differently. Provider coupling and substantial SQL logic are accepted costs.

Why not build a separate backend service?

Short answer: "I did not need an extra application server merely to proxy database reads. I put multi-row invariants in SQL transactions and used server functions where privileged work or a provider key justified them."

Deeper: the client still has explicit repository contracts. A custom API would add deployment, authentication integration, monitoring and another network boundary. **Follow-up:** when would you add one? Multiple independent clients, integration complexity or team ownership could justify it. Database constraints would remain.

Was it overengineered for personal use?

Short answer: "Some foundations were useful even for one person: account ownership, recoverable saves and conflict detection. Losing a response does not require a large user base. But custom biometric credentials and synchronization are significant complexity, and I would reevaluate standard auth and offline scope before expanding them."

Deeper: keep ownership/retries separate from capacity infrastructure. **Follow-up:** what was deferred? Server reminders, broad offline support, a web companion and distributed infrastructure. Do not claim that every implemented mechanism was the minimum possible design.

Answer drafts reflect stated motivations and inspected code. Adapt them to work you can personally demonstrate; they do not claim undocumented evaluations.

57. Interview answers: correctness and security

What if a meal saves but its response is lost?

Short answer: "The client persists an operation ID and frozen request. The database commits the meal and its receipt together. Retrying that same ID returns the result rather than creating another meal. A deliberately new identical meal gets a new ID."

Deeper: changed content cannot reuse the operation; accepted draft state survives until durable draft clearing.

Follow-up: exactly once? The network is not exactly-once; the mutation effect is deduplicated. AI calls and files are outside that transaction.

What if two devices edit the same record?

Short answer: "Each edit carries the version it read. If another device saved first, the stale edit gets a conflict and keeps its input for review. Workout replacement updates the session and sets in one transaction."

Deeper: goal changes compare selected fields so independent edits can merge. **Follow-up:** why not last-write-wins? It silently loses work. The cost is conflict UI and extra metadata; real simultaneous-connection testing is still needed beyond isolated SQL.

How do you isolate users?

Short answer: "The backend derives identity from authentication, enforces account policies on reads and checks active identity/ownership inside privileged mutation functions. Same-owner relationships prevent linking one account's child row to another account's parent."

Deeper: private object paths also have Storage policies; service-role keys stay server-side. **Follow-up:** are client filters enough? No; malicious requests can bypass the UI. Test two accounts directly. Ordinary read tokens and signed URLs can outlive credential revocation until expiry.

How does Face ID establish access?

Short answer: "Unlocking an existing privacy gate is separate from signing in. For fresh sign-in, Face ID releases a device-only protected secret, the server verifies and rotates it, and the client stores the replacement before accepting the new Auth session."

Deeper: server stores a hash and fixed expiry; remember-me stores identity, not a password. **Follow-up:** lost response? Password login/reenrollment is the fail-closed recovery. This custom protocol is a maintenance risk worth comparing against supported standards.

What actually works offline?

Short answer: "Manual vitals have a durable local cache and outbox transaction. Workouts, meals and cardio preserve drafts and retry state, but need server confirmation for completion. Cold-start authentication also needs connectivity."

Follow-up: why a cursor? It preserves an ordered, bounded change traversal; local page and cursor commit together. Tombstones stop stale clients resurrecting deleted readings.

58. Interview answers: features and growth

How do you keep AI output out of saved history?

Short answer: "The model proposes structured content. I validate it, recompute deterministic portions in code and keep the user in control. Meal estimates require review; workout generation fills editable drafts. Normal tracker saving still records completion."

Deeper: bounded context, consent, quotas and no mutation tools limit scope. **Follow-up:** does valid JSON mean accuracy? No. Representative meal/plan evaluation remains necessary; mock tests establish contract behavior, not quality.

Why snapshots and derived streaks?

Short answer: "A corrected label should not rewrite what a past meal meant. Entries keep reviewed nutrition snapshots. Streaks recalculate from dated targets and saved evidence so edits and backdated logs can change the result accurately."

Deeper: unknown history remains unknown; an under-calorie check does not prove full intake. **Follow-up:** why not store a counter? A counter needs invalidation for every edit/delete/rule change. Recalculation costs more reads and memory as history grows.

How would you deploy for others?

Short answer: "I would keep the managed architecture, separate staging and production, validate native recovery and account isolation, configure real email and privacy/support information, then use TestFlight before public release. Mobile binaries, database migrations and Edge Functions are separate releases."

Deeper: current remote updates are disabled; old app versions must remain compatible with new backend contracts. **Follow-up:** rollback? Prefer additive migrations and forward fixes, retain server artifacts, and test restore including photo bytes and client cursors.

What would break first as usage grows?

Short answer: "I would first measure long-history client work, export memory, per-account lock waits, photo bandwidth and AI cost. I have not established maximum capacity, and account count alone does not identify the bottleneck."

Deeper: test actions/day, history depth and concurrent bursts separately. **Follow-up:** microservices or Redis? Only when a measured need justifies their operating and consistency costs. Bounded queries and local normalization are more plausible first steps.

What would you improve with more time?

Short answer: "I would repair export completeness, complete native and real-concurrency acceptance, review custom auth/recovery storage, add photo reconciliation and test backups. Then I would profile long histories and build a representative AI evaluation set."

Follow-up: how measure success? Reproduce failure cases, verify no lost/duplicate records or cross-account access, set latency/memory budgets, and report actual outcomes rather than hypothetical scale.

59. Glossary: A-I

Access token: a signed credential attached to authenticated requests. It has a lifetime; ending a login preference does not necessarily recall every already-issued token.

ADR (Architecture Decision Record): a repository note explaining a selected approach and consequences. Amendments can supersede an original paragraph.

API (application programming interface): a defined contract for inputs and results between components. Sustain uses HTTP services and named SQL functions.

Atomic / atomicity: all parts of one transaction commit or none do. Local cache/outbox atomicity is separate from remote data/receipt atomicity.

Authentication / authorization: proving identity versus deciding permitted actions. A valid identity still needs ownership checks.

Backend / frontend: remote coordinating services versus the application interface and client logic. Some domain rules run in both for different reasons.

Cache: a reusable copy for availability or speed. It is not always authoritative and needs a refresh/reconciliation policy.

Capability: a secret whose possession permits a bounded action, such as a signed photo URL or deletion continuation token.

CAS (compare-and-set): apply a change only if the stored value/version still matches the expected baseline.

Concurrency: overlapping work, such as two phones saving the same record. It exists even with one human user.

Constraint: a database-enforced rule on valid stored data. A client-side form check is not a replacement.

Cursor: a remembered position in an ordered traversal. Vital pages and cursor advancement commit together locally.

Deployment: making code/configuration available in an environment. Uploading a mobile binary and migrating a database are different deployments.

Draft: unfinished local input. Its existence does not establish completed activity in cloud history.

Edge Function: on-demand hosted server code. It reduces server provisioning work but still needs authentication, resource limits and monitoring.

Foreign key: a database relationship that checks a referenced row exists; same-owner constraints additionally protect account relationships.

Idempotency: repeated delivery of the same identified operation does not repeat its accepted effect. A new operation with identical values is different.

Index: a database lookup structure that trades storage/write maintenance for faster selected reads. Plans and workload determine its value.

Definitions are tied to Sustain's implementation; pages 5-11 and 36-39 explain their practical context.

60. Glossary: J-Z

JSON / JWT: JavaScript Object Notation is a structured data format; a JSON Web Token is a signed token format. Plain JSON data is not authentication.

Keyset pagination: retrieve records after an immutable ordering key, rather than skipping a numeric offset that can shift after deletion.

Local notification: a request scheduled with the device OS. Remote push starts at a server and requires a separate delivery architecture.

Migration: an ordered schema/data change script. A file in Git is not evidence that a hosted database applied it.

Object storage: storage for file bytes addressed by paths/keys. Photo metadata transactions do not automatically include those bytes.

Observability: signals that explain system health and failures. Useful metrics avoid logging private content or credentials.

Optimistic concurrency: edit without a long-held lock, then check the version at save time and report conflicts.

Outbox: durable local work awaiting delivery. It is more than an in-memory retry timer.

p95: the 95th percentile of measured values, such as latency. Five percent may be slower; it is not a worst-case guarantee.

RLS (row-level security): database policies controlling which rows a role/user can access. Elevated server functions need their own checks.

RPC (remote procedure call): invoke a named operation remotely. Sustain's SQL mutation RPC groups transactional work.

RPO / RTO: recovery point objective limits tolerable data loss; recovery time objective limits tolerable restoration delay.

Saga: a multi-step workflow with explicit recovery across systems that cannot share one transaction, such as account deletion.

Schema: a specification of allowed structure. TypeScript, Zod and PostgreSQL enforce structure at different boundaries.

Session: the currently authenticated relationship with the service. Sustain's ordinary session is process-local.

Snapshot: a stored copy preserving meaning at a point in time, such as reviewed nutrition or a dated goal. It is not merely a stale cache.

Source of truth: the authoritative place for a defined category of information. Drafts, cloud facts and derived totals have different authority.

Tombstone: retained deletion knowledge that stops a stale client from restoring removed data accidentally.

Transaction: database changes committed or rolled back together. It does not automatically include files, OS actions or external calls.

Version / Zod: a version identifies a row revision for conflict detection; Zod validates actual runtime values rather than relying only on compile-time types.

61. Evidence map: application code

Paths are relative to the repository at 088f60c. Under the shorthand **src/**, use apps/mobile/src/. Feature directory references include their model, repository and adjacent tests; entry points below identify where to begin tracing the claim.

C01 - Stack and release config: root and apps/mobile/package.json, package-lock.json, apps/mobile/app.config.ts, app.json, eas.json. Verifies declared dependencies, native permissions, identities and disabled remote updates.

C02 - Decision/evidence discipline: AGENTS.md, docs/STATUS.md, ROADMAP.md, IMPLEMENTATION_LOG.md, docs/decisions/ADR-001 through ADR-010. Read amendments with their original decision.

C03 - Reachability: apps/mobile/app/_layout.tsx, app/index.tsx, (app)/_layout.tsx, (app)/track.tsx, (app)/coach.tsx, (app)/placeholder.tsx, (app)/profile/*, (auth)/*.

C04 - Service/local adapters: src/lib/supabase.ts, secure-store.ts, config.ts, pagination.ts, platform-specific storage files. Confirms process-only sessions, fallback behavior and page traversal.

C05 - Entry/auth interface: src/features/auth/auth-provider.tsx, auth-screen.tsx, auth-callback.ts, entry-state.ts, entry-provider.tsx:46, sustain-brand.tsx, launch-cover.tsx and private layout gates.

C06 - Biometric/security: src/features/auth/biometric-auth.ts, biometric-credential.ts, biometric-function-error.ts, security-settings.tsx; supabase/functions/biometric-auth/index.ts and _shared/biometric-*.

C07 - Setup/Profile: src/features/onboarding/{provider,repository,onboarding-screen,model}.*; src/features/profile/{profile-landing,personal-details,goal-editor,pending-changes,settings-ui}.*; shared UI sheets/input/date components.

C08 - Client consistency: src/lib/mutation-model.ts, mutations.ts:27 (sendMutation), mutations.ts:116 (runner), src/features/vitals/{sqlite-store,store-model,sync,storage.native,storage.web}.*.

C09 - SQL authority: supabase/migrations/202609180003_biometric_hardening.sql, 202609180004_consistent_writes.sql:62/78 (change reader/mutation RPC), 202609180005_remaining_ownership.sql, 202609210001_account_onboarding.sql, 202609220001_summary_refinements.sql, 202609220002_account_deletion.sql, plus earlier domain migrations.

C10 - Summary: src/features/summary/layout.ts (registry/defaults), data.ts:31 (allRows), dashboard.tsx, editor-grid.tsx, widget-renderers.tsx, extra-widgets.tsx, training.ts.

C11 - Streaks: src/features/streaks/{engine,model,repository,initialize}.*, src/features/summary/streak-week.ts; migrations 202609180001/002 and 202609220001.

The companion evidence manifest records source-file SHA-256 hashes. References identify reproducible source, not a live deployment attestation.

62. Evidence map: features and checks

C12 - Vitals: `src/domain/vitals.ts; src/features/vitals/{manual-vitals-log,blood-pressure,blood-pressure-pulse,chart,trend-card}.*`. **C13 - HealthKit:** `src/features/healthkit/{reader.ios,sync,identity,mapping,permissions,state,unified-sync}.*`.

C14 - Training/food/AI: `src/features/training/repository.ts:42/98/145, draft/progression modules; src/features/nutrition/{model,repository,draft,recipe,recipe-manager,ai-meal,ai-meal-client,ai-meal-editor}.*; src/features/coach/{repository,draft-actions,workout-questionnaire}.*; app/(app)/coach.tsx`.

C15 - Goals/fluids: `src/features/goals/{calculator,helper-model,helper-draft,goal-helper,repository}.* (saveGoalCalculation, line 237); src/features/profile/goal-display.ts; hydration model/repository/categories; supabase/functions/_shared/hydration.ts`.

C16 - Reminder UI/data: `src/features/reminders/{reminders-screen,custom-editor,routine-editor,routine-model,routine-storage,model,repository}.*`. **C17 - Scheduling/navigation:** `service.ts:223/298/309, schedule-model.ts, device.native.ts, notification-observer.tsx, navigation-model.ts, intent-storage.ts, lifecycle.ts` in that directory.

C18 - Photos: `src/features/progress-photos/{image,model,repository,progress-photo-gallery}.*`; photo handling in `src/lib/mutations.ts`; migrations `202609040002, 202609160001, 202609180005`.

C19 - Export: `src/features/data-export/repository.ts:15/48/96, model.ts, archive.native.ts, archive.web.ts`. **C20 - Deletion:** `src/features/auth/account-deletion.tsx, supabase/functions/delete-account/{index,cleanup}.ts, migration 202609220002`.

C21 - Server integrations: `supabase/functions/{estimate-meal,coach-chat,resolve-food-barcode,biometric-auth,delete-account}` and `_shared`; `docs/AI_MEAL_SETUP.md, WORKOUT_PLANNER_EVIDENCE.md, HARDENING.md`. Historical deployment notes are not fresh verification.

C22 - Fresh logs: `dist/sustain-system-review-check.log (240 app/66 Edge, lint/types); dist/sustain-system-review-sql.log (36 migrations/assertions); under apps/mobile/dist/, system-report-browser.log (40 passed), system-report-features.log (39 passed/6 failed), system-report-rerun.log (3 passed/5 failed), system-report-reduced.log (8 passed with Reduce Motion)`.

The five frozen-clock failures pass through the existing Reduce Motion path, supporting an animation-clock/harness interaction rather than failure of their later date assertions. That diagnostic does not certify frozen-clock animated entry. The obsolete Soon-label assertion remains a test-maintenance failure. No app or original test code was changed.

SQL command: `node scripts/check-onboarding-sql.mjs`. Browser suites cover `account/entry/onboarding/reminders/Profile/deletion` and `AI/food/drinks/dates/helpers/photos/recipes/Summary`. All use synthetic fixtures. Historical native export/config and hosted deployment records remain separately labeled in STATUS/ADRs.

63. Authoritative references: platform

Official pages were checked on **23 September 2026**. Platform documentation supports platform claims; application code supports Sustain-specific behavior. Requirements can change, so recheck at release time. No quoted pricing is used.

R01 - Supabase production checklist. Ownership, production configuration and release preparation. [Production checklist](#).

R02 - Supabase shared responsibility. Managed infrastructure does not remove the app owner's responsibilities for queries, access, resources and configuration. [Shared responsibility model](#).

R03 - Supabase row-level security. Database policy semantics and elevated-role considerations. [Row-level security](#).

R04 - PostgreSQL transaction isolation. Concurrency/isolation definitions; Sustain's extra version and lock protocol is established by its own SQL. [Transaction isolation](#).

R05 - Expo build setup. Native build preparation and credentials. [Create your first build](#).

R06 - Expo iOS submission. Uploading an iOS build through EAS Submit is a distribution step, not store approval. [Submit to the Apple App Store](#).

R07 - Supabase SMTP. Default email restrictions and custom sender configuration. [Send emails with custom SMTP](#).

R08 - Apple TestFlight. Beta distribution, external review and build lifetime. [TestFlight overview](#).

R09 - Apple SDK minimum. Published submission requirement effective April 28, 2026. [SDK minimum requirements](#).

R10 - Apple HealthKit privacy. Permission/read-privacy model; empty reads cannot simply be equated with permission denial. [Protecting user privacy](#).

R11 - Apple local notifications. The OS schedules delivery from content and trigger requests. This is not proof of actual delivery on a tested phone. [Scheduling a notification locally](#).

No platform source certifies this application's deployment, data isolation, supported user count, or regulatory status.

64. References and reproducibility

R12 - Supabase Storage access. Private-object policies and authorization. [Storage access control](#).

R13 - Supabase backups. Database backups exclude Storage object bytes. [Database backups](#).

R14 - Open Food Facts API. Integration, rate limits and license/attribution guidance. [API documentation](#).

R15 - Mifflin et al., 1990. Original resting-energy prediction equation, not a validation of every activity multiplier or loss scenario used by Sustain. [Research abstract](#), DOI 10.1093/ajcn/51.2.241.

R16 - Institute of Medicine, 2005. Total-water and beverage reference distinctions. [Dietary Reference Intakes, Water chapter](#). The app's activity additions remain product assumptions.

R17 - Apple account deletion. Requirements for account-creating applications. [Offering account deletion in your app](#).

R18 - Apple privacy disclosures. Include relevant third-party practices and keep responses accurate. [App privacy details](#).

R19 - Apple review guidelines. Privacy policy, review access, accurate metadata and health-data use restrictions. [App Review Guidelines](#).

R20 - Expo SDK 57 updates. Remote-update configuration and native runtime compatibility. [Expo Updates](#). Sustain disables this release path in its current config.

R21 - Supabase Edge limits. Hosted memory, CPU, duration and idle limits are distinct from app-level provider timeouts. [Edge Function limits](#).

Reproducing the review

Use application commit 088f60c, install its locked dependencies, and run the commands indexed on page 62 in isolated environments. The accompanying source manifest records hashes for relevant source/config/test files and the supplied reference PDF. The PDF, Markdown and HTML are documentation outputs; scratch render/test diagnostics are excluded from app dependencies.

All pages were rendered and checked for page count, extracted text, layout bounds, diagrams, links and footer separation. The report establishes a review of available evidence, not live service attestation. Native behavior, real concurrency, current hosted configuration, paid AI quality and maximum capacity remain explicit next verification work.